

Công nghệ phần mềm

Chương 06: Đảm bảo chất lượng, kiểm thử và bảo trì phần mềm

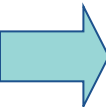


Giảng viên: Lê Thị Hoàng Anh

Email: anhlt@nuce.edu.vn



Nội dung

- 
1. Đảm bảo chất lượng phần mềm
 2. Kiểm thử phần mềm
 3. Bảo trì phần mềm



Đảm bảo chất lượng phần mềm

- Liên quan đến việc đảm bảo một sản phẩm phần mềm đạt được mức độ yêu cầu về chất lượng
- Liên quan đến việc định nghĩa các tiêu chuẩn chất lượng phù hợp và đảm bảo rằng những điều này được tuân thủ.



Các nhân tố chất lượng phần mềm

- Là cái mốc để đánh giá chất lượng phần mềm. Tập trung vào ba khía cạnh:
 - Các đặc trưng vận hành
 - Khả năng trải qua thay đổi
 - Tính thích nghi với môi trường mới
- Một sản phẩm phần mềm cần xét theo các tiêu chuẩn:
 - Tính đúng đắn, tính khoa học, tính hữu hiệu, tính sáng tạo, tính an toàn, tính toàn vẹn, tính đầy đủ, tính độc lập, tính phổ dụng, tính đơn giản, tính dễ phát triển để mở rộng...



Độ đo chất lượng phần mềm

- Các độ đo phần mềm được dùng cho việc thẩm định mang tính định lượng:
 - Chỉ số chất lượng phần mềm
 - Khoa học phần mềm HALSTEAD
 - Độ đo phức tạp của Thomas McCabe



Độ tin cậy phần mềm

- Độ tin cậy phần mềm có thể được đo trực tiếp và ước lượng bằng cách dùng dữ liệu lịch sử và đang phát triển.
- Được định nghĩa dưới dạng xác suất thống kê:
 - Số lần vận hành không thất bại/ tổng số lần
 - Xét trong một khoảng thời gian xác định



Độ tin cậy phần mềm

- Cách đo đơn giản về độ tin cậy là thời gian trung bình giữa những lần thất bại:

$$MTBF = MTTF + MTTR$$

– Trong đó:

- MTTF là thời gian trung bình của thất bại
- MTTR là thời gian trung bình để sửa chữa tương ứng



Độ tin cậy phần mềm

- Ngoài độ tin cậy chúng ta cần đo tính có sẵn của phần mềm.
 - Là xác suất một chương trình vận hành theo yêu cầu tại một điểm nào đó theo thời gian

$$\text{Tính sẵn có} = \frac{MTTF}{(MTTF + MTTR)} \times 100\%$$



Cách tiếp cận đảm bảo chất lượng phần mềm

- Đảm bảo chất lượng phần mềm SQA là một mẫu hình hành động có hệ thống và có kế hoạch.
- Bao gồm 7 hoạt động:
 - (1) Áp dụng các phương pháp kỹ thuật, (2) Tiến hành các xét duyệt kỹ thuật chính thức, (3) Kiểm thử phần mềm, (4) Buộc tôn trọng các chuẩn, (5) Kiểm soát thay đổi, (6) Đo, (7) Lưu giữ và báo cáo kết quả ghi lại.



Hoạt động quản lý chất lượng

1. Quality Assurance (QA)

- Thiết lập quy trình và tiêu chuẩn chất lượng

2. Quality Planning (QP)

- Lựa chọn các thuộc tính chất lượng

3. Quality Control (QC)

- Đảm bảo các quy trình và tiêu chuẩn được tuân thủ

Quản lý chất lượng nên tách biệt với quản lý dự án để đảm bảo sự độc lập



Nội dung

1. Đảm bảo chất lượng phần mềm
- 2. Kiểm thử phần mềm
3. Bảo trì phần mềm



Một số định nghĩa về kiểm thử

- Kiểm thử phần mềm là quá trình thực thi phần mềm với mục tiêu tìm ra lỗi

Glen Myers, 1979

→ Khẳng định được chất lượng của phần mềm đang xây dựng

Hetzel, 1988

- Kiểm thử là một phần của quy trình thẩm định và kiểm định phần mềm (verification and validation – V&V).



Một số đặc điểm của kiểm thử PM

- Kiểm thử phần mềm giúp tìm ra được sự hiện diện của lỗi nhưng không thể chỉ ra sự vắng mặt của lỗi

Dijkstra

- Mọi phương pháp được dùng để ngăn ngừa hoặc tìm ra lỗi đều sót lại những lỗi khó phát hiện hơn

Beizer

- Điều gì xảy ra nếu việc kiểm thử không tìm được lỗi trong phần mềm hoặc phát hiện quá ít lỗi
 - Phần mềm có chất lượng quá tốt
 - Quy trình/Đội ngũ kiểm thử hoạt động không hiệu quả



6 điểm lưu ý khi kiểm thử

- (1) Chất lượng phần mềm do khâu phân tích, thiết kế quyết định là chủ yếu chứ không phải khâu kiểm thử.
- (2) Tính dễ kiểm thử phụ thuộc vào cấu trúc chương trình
- (3) Người kiểm thử và người phát triển nên khác nhau



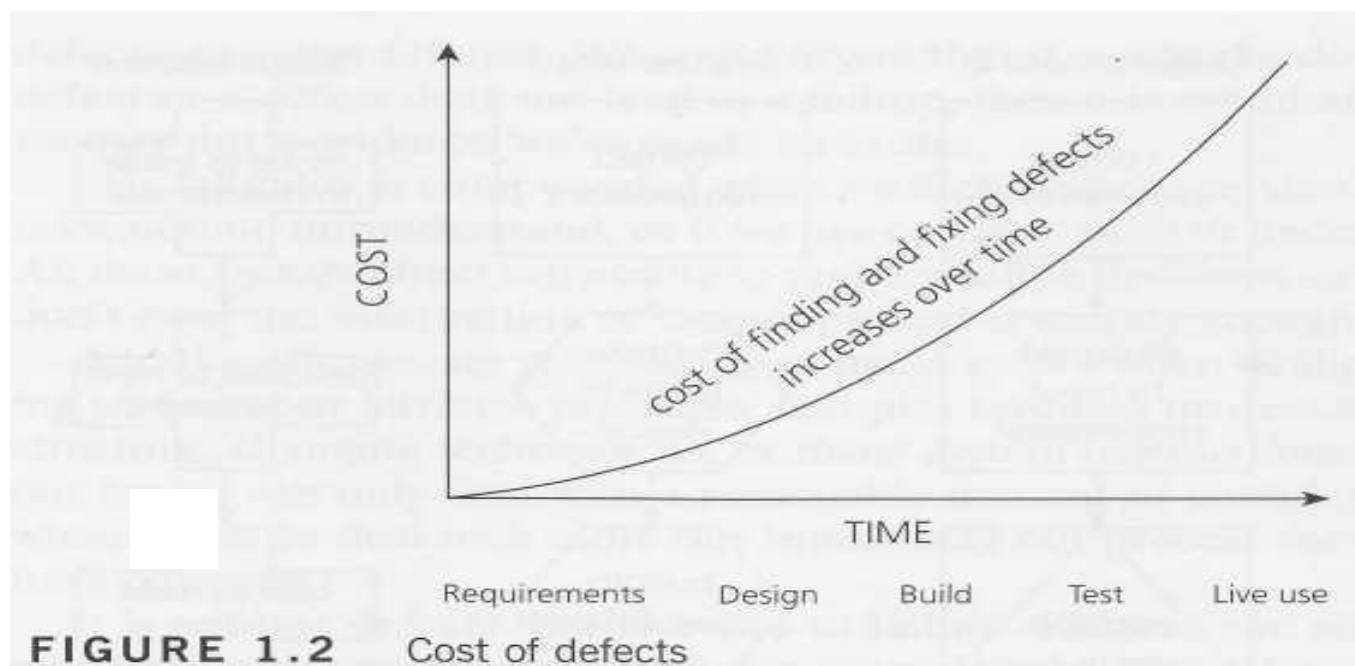
6 điểm lưu ý khi kiểm thử (tiếp)

- (4) Dữ liệu thử cho kết quả bình thường thì không có ý nghĩa nhiều, cần có những dữ liệu kiểm thử mà phát hiện ra lỗi
- (5) Khi thiết kế trường hợp thử, không chỉ dữ liệu kiểm thử nhập vào, mà phải thiết kế trước cả dữ liệu kết quả sẽ có
- (6) Khi phát sinh thêm trường hợp thử thì nên thử lại những trường hợp thử trước đó để tránh ảnh hưởng lan truyền sóng



Chi phí tìm lỗi tăng lên khi nào?

- Chi phí cho việc tìm thấy và sửa lỗi **tăng dần** trong suốt chu kỳ sống của phần mềm. Lỗi tìm thấy **càng sớm** thì **chi phí để sửa càng thấp** và ngược lại.





Thời điểm tiến hành kiểm thử

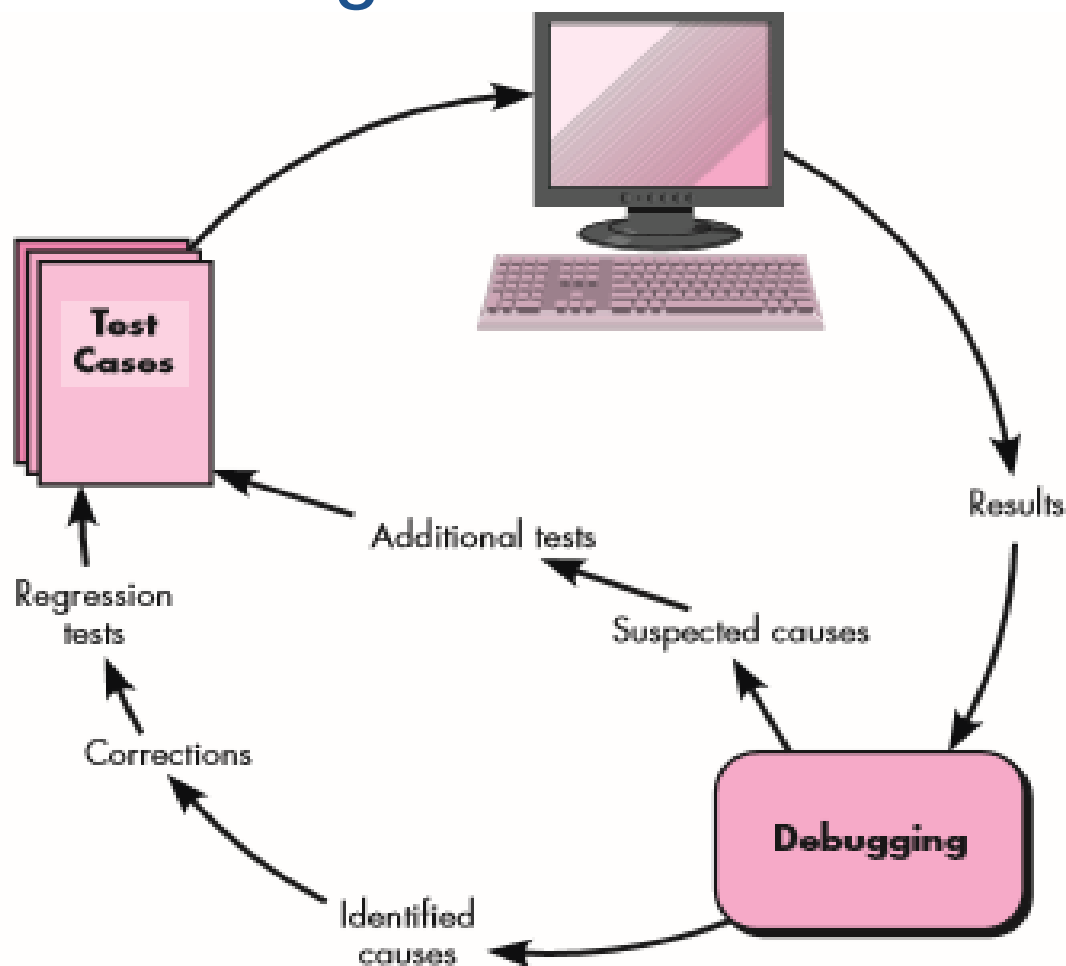
Tiến hành ở mọi công đoạn phát triển phần mềm

- ❑ phân tích
 - xét duyệt đặc tả yêu cầu
- ❑ thiết kế
 - xét duyệt đặc tả thiết kế
- ❑ mã hóa
 - kiểm thử chương trình



Phân biệt debug và testing

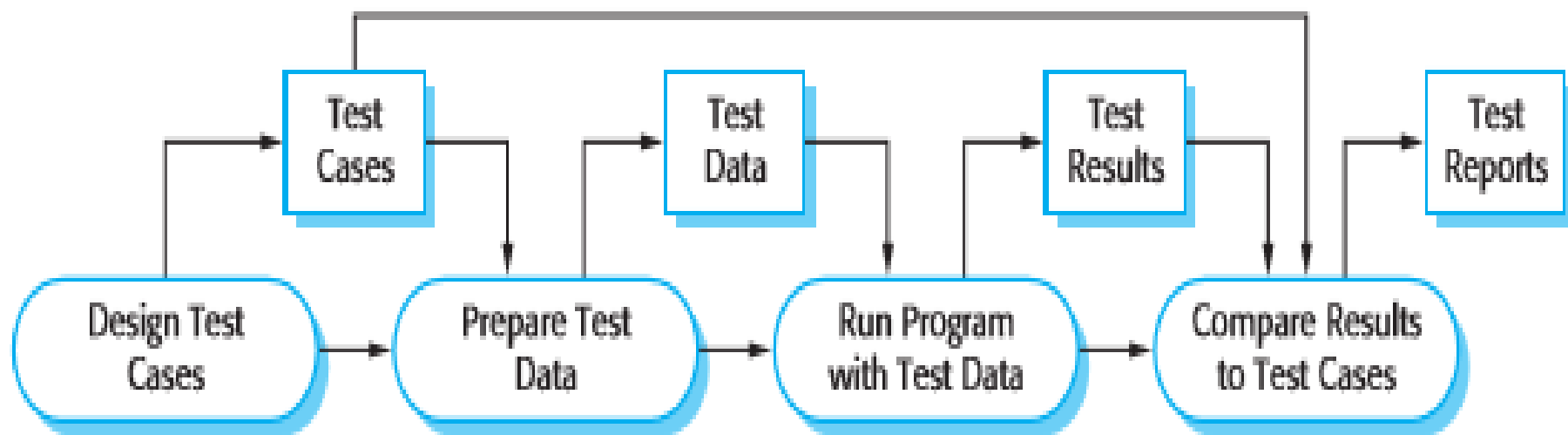
- Quá trình debug





Phân biệt debug và testing

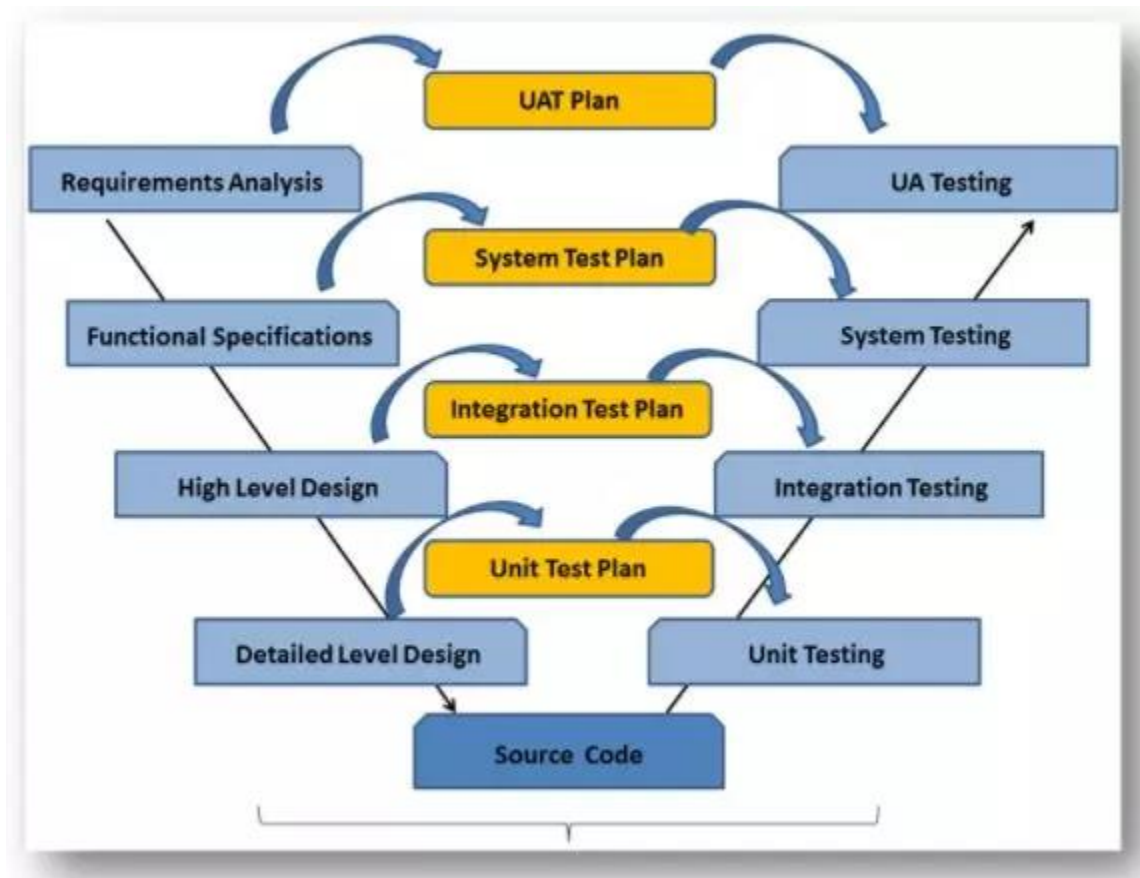
Một mô hình quy trình kiểm thử phần mềm





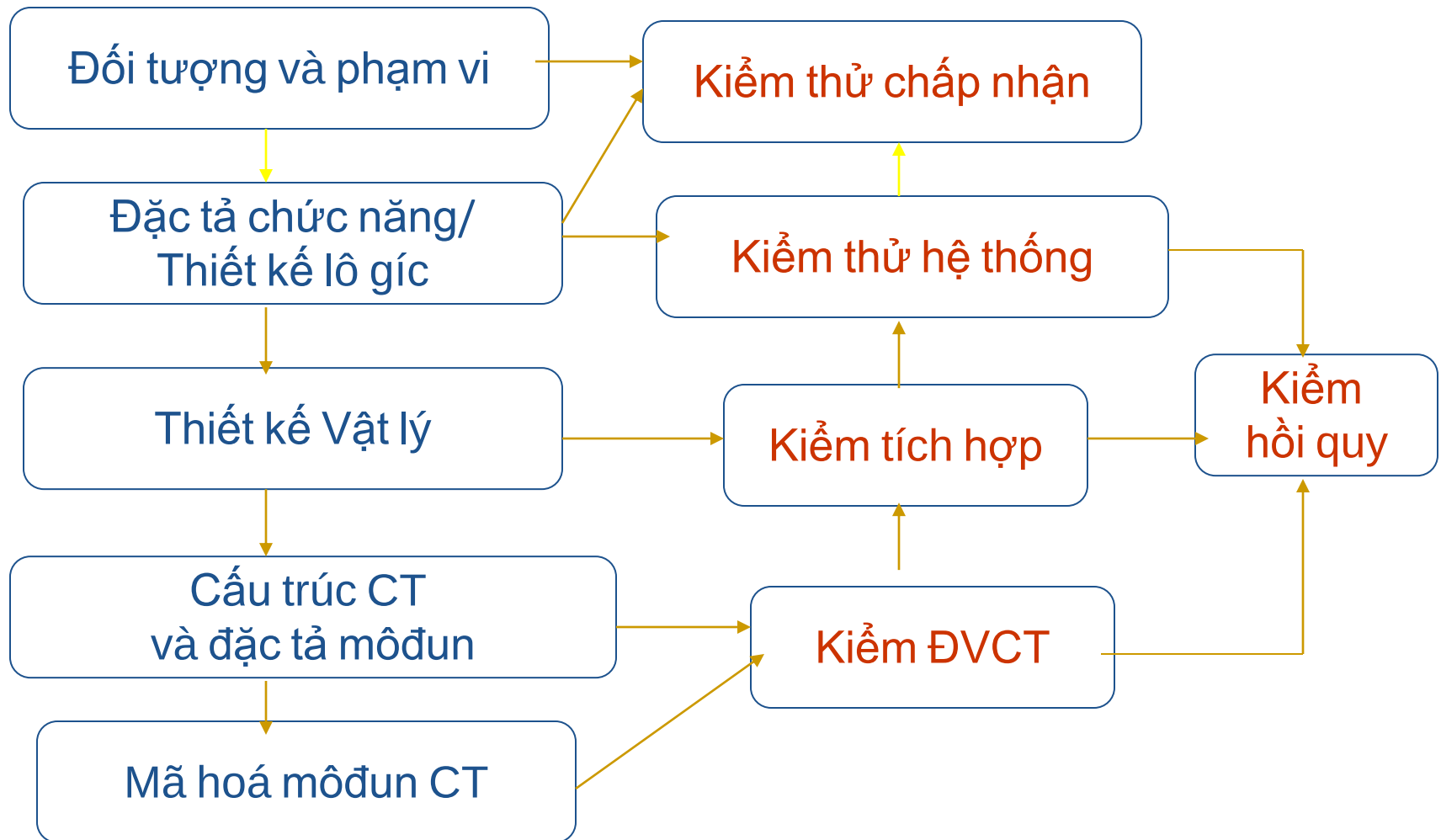
Tương ứng giữa vòng đời dự án và kiểm thử

- Mô hình chữ V





Tương ứng giữa vòng đời dự án và kiểm thử





Nguyên tắc kiểm thử

- Chọn các input làm cho hệ thống tạo ra tất cả các thông báo lỗi;
- Thiết kế input làm tràn bộ đệm;
- Lặp lại cùng một input hay một dãy các input một vài lần;
- Ép các output không hợp lệ phải xuất hiện;
- Ép các kết quả tính toán phải hoặc là quá lớn hoặc là quá nhỏ.



Quan điểm kiểm thử

- Kiểm thử chỉ ra sự hiện diện của lỗi
- Kiểm thử toàn diện đầy đủ là không thể
- Cần bắt đầu giai đoạn kiểm thử càng sớm càng tốt
- Phân nhóm lỗi để xác định một số module tập trung lỗi nhiều nhất
- Pesticide paradox
- Kiểm thử được thực hiện khác nhau trong những bối cảnh khác nhau
- Suy nghĩ không có lỗi là một sai lầm



Chính sách kiểm thử (Testing Policy)

- Kiểm tra tất cả các chức năng trong hệ thống menu.
- Kiểm tra tất cả các **mục khác** có cùng chức năng trong hệ thống menu (Toolbar, Listbar, Dialog bar, Context Menu,...)
- Kiểm tra cùng một chức năng với nhiều vai trò khác nhau (đối với hệ thống có nhiều người dùng)
- Kiểm tra tất cả các **dữ liệu bắt buộc nhập** trong các màn hình (hợp lệ/không hợp lệ)



Các giai đoạn của kiểm thử

- Kiểm thử trong khi xây dựng (Development testing)
Hệ thống được kiểm thử trong quá trình phát triển để tìm ra lỗi.
- Kiểm thử bản release (release testing)
Một đội ngũ kiểm thử độc lập sẽ kiểm thử một phiên bản hoàn chỉnh của hệ thống trước khi nó được bàn giao cho người dùng.
- Kiểm thử người dùng (user testing)
Người dùng hoặc người dùng tiềm năng của hệ thống kiểm thử hệ thống trên môi trường của họ



Phân loại

- Kiểm thử trong khi xây dựng: Gồm tất cả các hoạt động kiểm thử được tiến hành bởi nhóm phát triển hệ thống.
 - Kiểm thử đơn vị (unit testing)
 - Kiểm thử component (component testing)
 - Kiểm thử tích hợp (intergration testing)
 - Kiểm thử hệ thống (system testing)
- Kiểm thử bản release: Kiểm thử để chuẩn bị phát hành(validation test và defect test).
 - Scenario based testing – kiểm thử theo kịch bản (use case)
 - Performance testing – kiểm thử hiệu năng
- Kiểm thử người dùng(user testing)
 - Kiểm thử người dùng gồm: Acceptance test, Alpha,Beta testing



Kiểm thử đơn vị trong OOP

- Là quy trình kiểm thử từng thành phần riêng lẻ.
- Đây là quy trình kiểm thử tìm lỗi.
- Các đơn vị có thể là:
 - Các hàm hay phương thức đơn lẻ trong một lớp đối tượng (class).
 - Các lớp đối tượng (class) chứa vài thuộc tính và phương thức.
 - Các thành phần với các giao diện được định nghĩa sẵn để truy cập vào các tính năng của chúng.



Kiểm thử đơn vị trong OOP

- Người tiến hành thường là người lập trình
- Sử dụng các stubs, mock, fake và drivers
- Phối hợp giữa kiểm thử cấu trúc và kiểm thử chức năng
 - Kiểm tra câu lệnh điều kiện, cấu trúc dữ liệu điều kiện biên,...
- Tài liệu thường sơ sài



Kiểm thử thành phần trong OOP (component testing)

- Kiểm thử thành phần là quá trình kiểm thử các thành phần riêng biệt của hệ thống.
- Với hầu hết các hệ thống, người phát triển các thành phần chịu trách nhiệm kiểm thử các thành phần.
- Ta có thể kiểm thử chúng theo các bước sau:
 1. Các chức năng và cách thức riêng biệt bên trong đối tượng.
 2. Các lớp đối tượng có một vài thuộc tính và phương thức.
 3. Kết hợp các thành phần **để tạo nên các đối tượng và chức năng khác nhau**. Các thành phần hỗn hợp có một giao diện rõ ràng được sử dụng để truy cập các chức năng của chúng.



Kiểm thử lớp đối tượng (Object class testing)

- Một test hoàn chỉnh cho một lớp bao gồm
 - Kiểm thử tất cả các phương thức liên kết với một đối tượng;
 - Thiết lập và thăm vấn tất cả thuộc tính của đối tượng;
 - Thực hành đối tượng tại tất cả trạng thái có thể.
- Tính kế thừa làm cho việc kiểm thử lớp đối tượng khó hơn bởi vì thông tin được kiểm thử không có tính cục bộ.



Kiểm thử tích hợp trong OOP

- Quá trình kiểm thử tích hợp bao gồm việc xây dựng hệ thống từ các thành phần và kiểm thử hệ thống tổng hợp với các vấn đề phát sinh từ sự tương tác giữa các thành phần.
- Kiểm thử tích hợp kiểm tra trên thực tế các thành phần làm việc với nhau, được gọi là chính xác và truyền dữ liệu đúng, vào lúc thời gian đúng, thông qua giao diện của chúng.
- Hệ thống tích hợp bao gồm một nhóm các thành phần thực hiện vài chức năng của hệ thống và được tích hợp với nhau bằng cách gộp các mã. Tích hợp theo hai cách là bottom-up hay top-down...



Kiểm thử tích hợp trong OOP

- Trong Unit Test, lập trình viên cố gắng phát hiện lỗi liên quan đến chức năng và cấu trúc nội tại của Unit. Có một số phép kiểm thử đơn giản trên giao tiếp giữa Unit với các thành phần liên quan khác, tuy nhiên mọi giao tiếp liên quan đến Unit chỉ thật sự được kiểm tra đầy đủ khi các Unit tích hợp với nhau trong khi thực hiện Integration Test.
- Trừ một số ít ngoại lệ, Integration Test chỉ nên thực hiện trên những Unit đã được kiểm tra cẩn thận trước đó bằng Unit Test, và tất cả các lỗi mức Unit đã được sửa chữa.



Kiểm thử tích hợp trong OOP

- Có 4 loại kiểm tra trong KTTH:
 - Kiểm tra cấu trúc
 - Kiểm tra chức năng
 - Kiểm tra hiệu năng
 - Kiểm tra khả năng chịu tải



Kiểm thử hệ thống

- Mục đích là kiểm thử thiết kế và toàn bộ hệ thống (sau khi tích hợp) có thỏa mãn yêu cầu đặt ra hay không.
- Trong nhiều trường hợp, đòi hỏi một số thiết bị phụ trợ, phần mềm hoặc phần cứng đặc thù.
- Với hầu hết các hệ thống phức tạp, kiểm thử hệ thống gồm hai giai đoạn riêng biệt:
 1. Kiểm thử tích hợp: đội kiểm thử nhận mã nguồn của hệ thống và nhận biết thành phần cần phải gỡ lỗi. Kiểm thử tích hợp hầu như liên quan với việc tìm các khiếm khuyết của hệ thống.
 2. Kiểm thử phát hành: Một phiên bản của hệ thống có thể được phát hành tới người dùng được kiểm thử, thường là kiểm thử “hộp đen”. Các vấn đề được báo cáo cho đội phát triển để gỡ lỗi chương trình. Khách hàng được bao hàm trong kiểm thử phát hành, thường được gọi là kiểm thử chấp nhận.



Kiểm thử bản release

- Là quy trình kiểm thử một bản release của hệ thống, bản này sẽ sử dụng bên ngoài đội ngũ phát triển hệ thống.
- Mục tiêu chính là để thuyết phục khách hàng rằng hệ thống đủ tốt để đưa vào sử dụng.
 - Phải chỉ ra được rằng hệ thống hỗ trợ các tính năng đã đặc tả, đảm bảo hiệu năng và độ tin cậy, và không có lỗi khi sử dụng.
- Là quy trình kiểm thử hộp đen trong đó các test chỉ bắt nguồn từ đặc tả hệ thống.



Kiểm thử bản release

- Kiểm thử bản release là một hình thức của kiểm thử hệ thống.
- Điểm khác nhau quan trọng:
 - Một nhóm tách biệt không tham gia vào việc phát triển sẽ chịu trách nhiệm về kiểm thử bản release.
 - Kiểm thử hệ thống bởi nhóm phát triển nên tập trung vào việc tìm lỗi trong hệ thống (defect testing).
 - Mục tiêu của kiểm thử bản release là để chứng tỏ rằng hệ thống đáp ứng yêu cầu và đủ tốt để đưa ra sử dụng bên ngoài (validation testing).



Kiểm thử người dùng user testing

- Acceptance Test – Kiểm thử chấp nhận: Test được thiết kế với sự giúp đỡ của người dùng
 - Truyền thống:
 - do khách hàng thực hiện
 - thực hiện thủ công
 - sau khi sản phẩm phần mềm được bàn giao
 - Dựa trên các use case / user story
 - Agile software development:
 - Kiểm thử tự động: JUnit, Fit, . . .
 - Test được tạo trước khi user story được cài đặt



Acceptance Test

- Có sự tham gia của khách hàng/người sử dụng
- Dùng kiểm thử chức năng
- Mục đích: thẩm định(validation) phần mềm – sai sót, thiếu sót - so với yêu cầu người dùng
- Sử dụng các dữ liệu thực do user cung cấp



Kiểm thử người dùng

- Alpha testing:
 - Kiểm thử chấp nhận tiến hành ở môi trường khách hàng được gọi là alpha testing
- Beta testing:
 - Mở rộng của alpha testing
 - Được tiến hành với một lượng lớn users
 - User tiến hành kiểm thử không có sự hướng dẫn của người phát triển
 - Thông báo lại kết quả cho người phát triển



Phân loại theo chiến lược kiểm thử (strategy)

- Kiểm nghiệm tầm hợp: Kiểm nghiệm các bộ phận riêng rẽ, bao gồm:

White-box testing :

- Component hay Unit Testing,
- Object class testing

Black-box testing:

- Functional testing, Interface testing, Alpha testing , Beta testing, Release testing, Stress testing, Ad hoc testing, Performance testing...

Gray-box testing:

- Trong Kiểm thử Hộp xám, cấu trúc bên trong sản phẩm chỉ được biết một phần, Tester có thể truy cập vào cấu trúc dữ liệu bên trong và thuật toán của chương trình với mục đích là để thiết kế các test case, nhưng khi test thì test như là người dùng cuối hoặc là ở mức hộp đen.



Phân loại theo chiến lược kiểm thử (strategy)

- Kiểm nghiệm tầm rộng:
 - Kiểm nghiệm bộ phận (Module testing): kiểm nghiệm một bộ phận riêng rẽ.
 - Kiểm nghiệm tích hợp (Integration testing): tích hợp các bộ phận và hệ thống con.
 - Kiểm nghiệm hệ thống (System testing): kiểm nghiệm toàn bộ hệ thống.
 - Kiểm nghiệm chấp nhận (Acceptance testing): thực hiện bởi khách hàng.



Một số kỹ thuật test

- Test tĩnh:
 - Dựa vào việc kiểm tra tài liệu, source code,... mà không cần phải thực thi phần mềm.
 - Các lỗi được tìm thấy trong quá trình kiểm tra có thể dễ dàng được loại bỏ và chi phí rẻ hơn nhiều so với khi tìm thấy trong test động. Một số lợi ích khi thực hiện việc kiểm tra (reviews):
 - Lỗi sớm được tìm thấy và sửa chữa
 - Giảm thời gian lập trình
 - Giảm thời gian và chi phí test



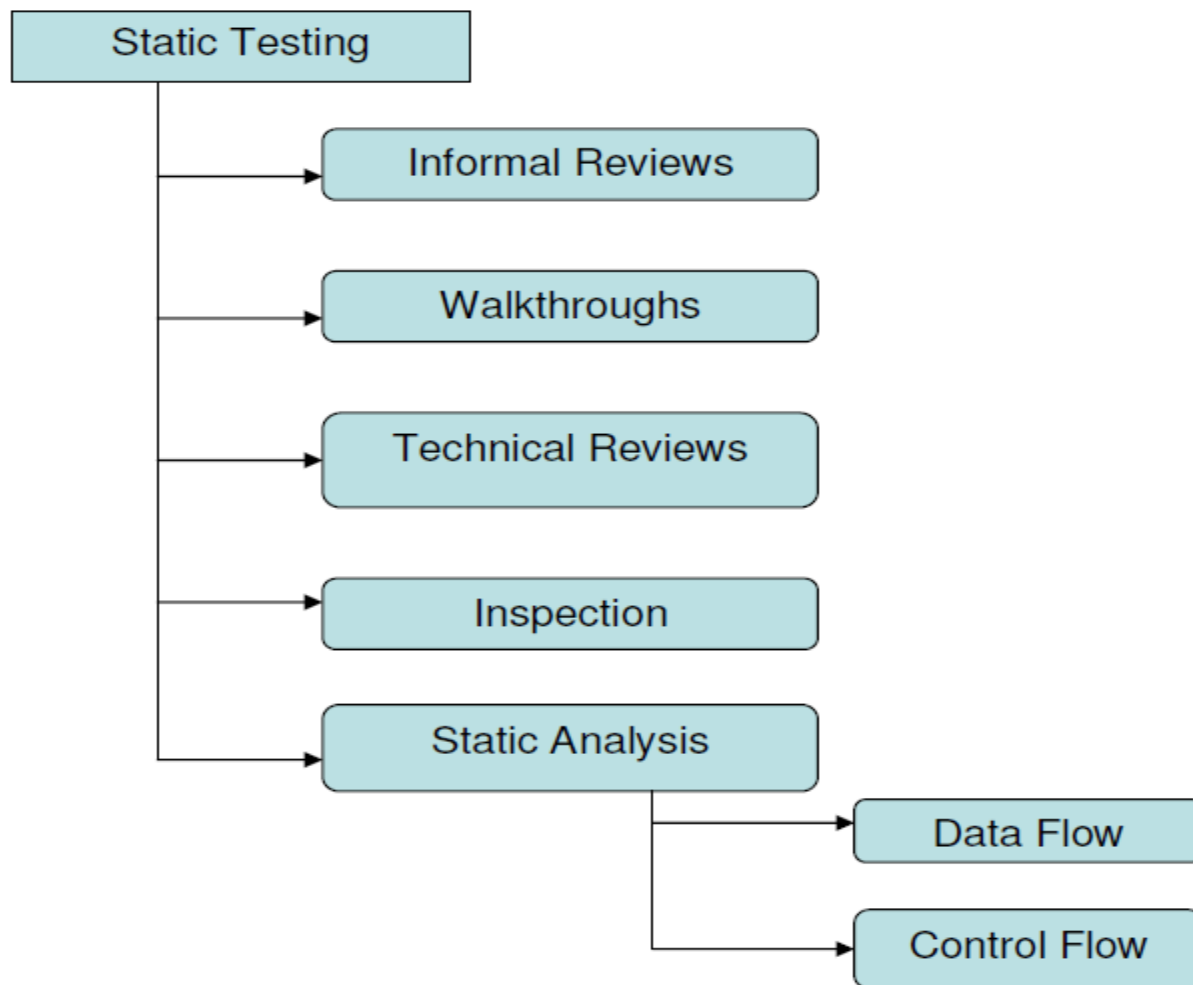
Một số kỹ thuật test

- Test tĩnh (tiếp):
 - Các tài liệu được kiểm thử:
 - Tài liệu đặc tả yêu cầu
 - Tài liệu đặc tả thiết kế
 - Sơ đồ luồng dữ liệu
 - Mô hình ER
 - Source code
 - Test case
 - ...



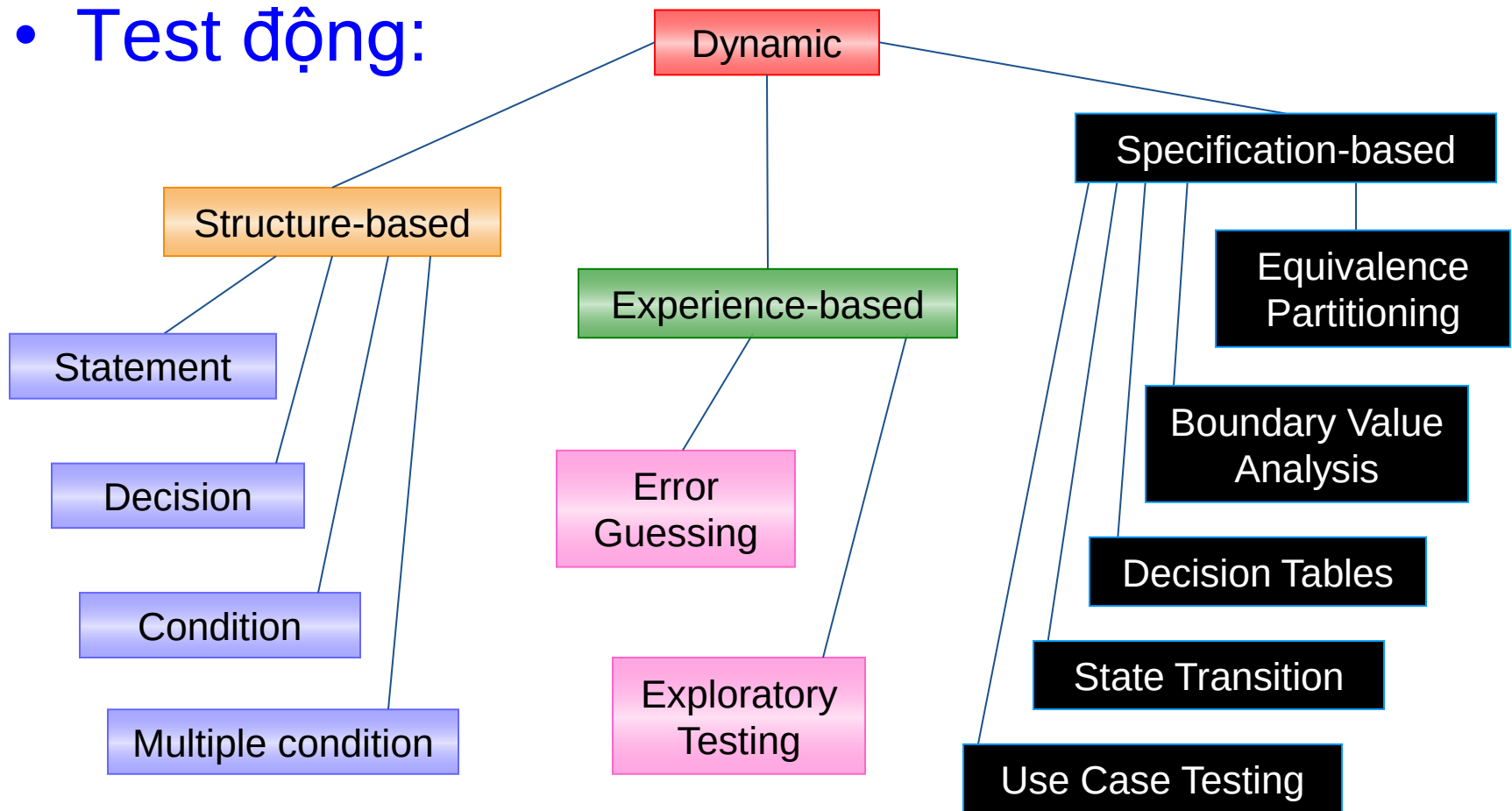
Các kỹ thuật trong kiểm thử tĩnh

- Tổng quan



Một số kỹ thuật test

- Test động:





Một số kỹ thuật test

- Test động:
 - Test dựa trên mô tả (specification-based) hay còn gọi test chức năng (functional testing): Test những gì mà phần mềm phải làm, không cần biết phần mềm làm như thế nào (kỹ thuật **black box**)
 - Test dựa trên cấu trúc (structure-based) hay còn gọi test phi chức năng (non-functional testing): Test phần mềm hoạt động như thế nào (kỹ thuật **white box**)
 - Test dựa trên kinh nghiệm (experience-based): đòi hỏi sự hiểu biết, kỹ năng và kinh nghiệm của người test



Các kỹ thuật trong kiểm thử hộp đen

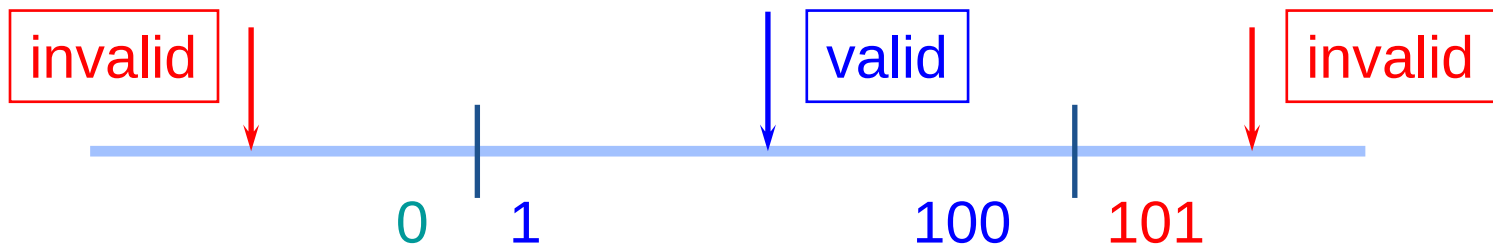
- Kỹ thuật phân vùng tương đương – EP (Equivalence Partitioning)
 - Ví dụ: một textbox chỉ cho phép nhập số nguyên từ 1 đến 100
 - → Ta không thể nhập tất cả các giá trị từ 1 đến 100
 - Ý tưởng của kỹ thuật này: chia (partition) đầu vào thành những nhóm tương đương nhau (equivalence). Nếu một giá trị trong nhóm hoạt động đúng thì tất cả các giá trị trong nhóm đó cũng hoạt động đúng và ngược lại.



Các kỹ thuật trong kiểm thử hộp đen

- Kỹ thuật phân vùng tương đương – EP (tiếp)

- Trong ví dụ trên dùng kỹ thuật phân vùng tương đương, chia làm 3 phân vùng như sau:



- Như vậy chỉ cần chọn 3 test case để test trường hợp này: -5, 55, 102 hoặc 0, 10, 1000, ...



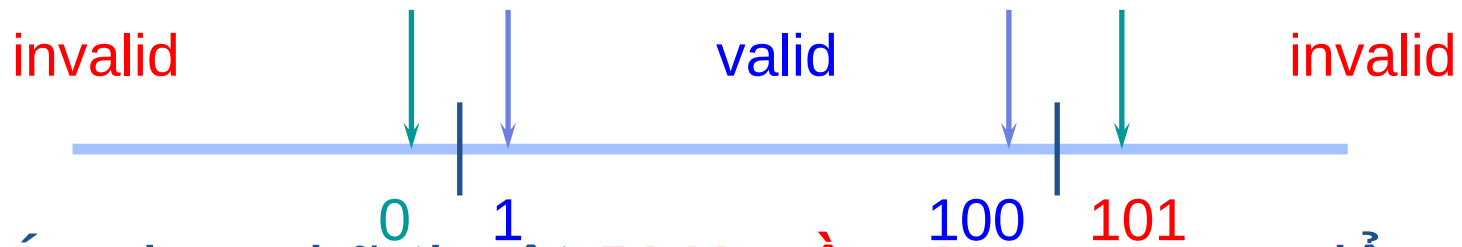
Các kỹ thuật trong kiểm thử hộp đen

- Kỹ thuật phân vùng tương đương – EP (tt)
 - Tuy nhiên nếu ta nhập vào **số thập phân** (55.5) hay một **ký tự không phải là số** (abc)?
 - Trong trường hợp trên có thể chia làm 5 phân vùng như sau:
 - Các số nguyên từ 1 đến 100
 - Các số nguyên nhỏ hơn 1
 - Các số nguyên lớn hơn 100
 - Không phải số
 - Số thập phân
 - Như vậy, việc phân vùng có đúng và đủ hay không là tùy thuộc vào kinh nghiệm của tester.



Các kỹ thuật trong kiểm thử hộp đen

- Kỹ thuật phân tích giá trị giới hạn - BVA (Boundary Value Analysis)
 - Kỹ thuật BVA sẽ chọn các giá trị nằm tại các điểm giới hạn của phân vùng.



- Áp dụng kỹ thuật BVA cần 4 test case để test trường hợp này: 0,1,100,101



Kỹ thuật EP & BVA

- Xét ví dụ: Một ngân hàng trả lãi cho khách hàng dựa vào số tiền còn lại trong tài khoản. Nếu số tiền từ 0 đến 100\$ thì trả 3% lãi, từ lớn hơn 100 \$ đến nhỏ hơn 1000\$ trả 5% lãi, từ 1000\$ trở lên trả 7% lãi.

– Dùng kỹ thuật EP:

Invalid partition	Valid (for 3% interest)		Valid (for 5%)		Valid (for 7%)
-\$0.01	\$0.00	\$100.00	\$100.01	\$999.99	\$1000.00

- Kỹ thuật EP: -0.44, 55.00, 777.50, 1200.00
- Kỹ thuật BVA: -0.01, 0.00, 100.00, 100.01, 999.99, 1000.00



Tại sao phải kết hợp BVA và EP

- Mỗi giá trị giới hạn đều nằm trong một phân vùng nào đó. Nếu chỉ sử dụng giá trị giới hạn thì ta cũng có thể test luôn phân vùng đó.
- Tuy nhiên vấn đề đặt ra là nếu như giá trị đó sai thì nghĩa là giá trị giới hạn bị sai hay là cả phân vùng bị sai. Hơn nữa, nếu chỉ sử dụng giá trị giới hạn thì không đem lại sự tin tưởng cho người dùng vì chúng ta chỉ sử dụng những giá trị đặc biệt thay vì sử dụng giá trị thông thường.
- Vì vậy, cần phải kết hợp cả BVA và EP



Ví dụ

Customer Name

2-64 chars.

Account number

6 digits, 1st
non-zero

Loan amount requested

£500 to £9000

☐ Term of loan

☐ Monthly repayment

1 to 30 years

Term:

Repayment:

Interest rate:

Total paid back:

Minimum £10

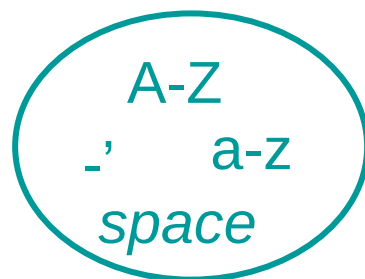


Customer name

Number of characters:



Valid characters:



Conditions	Valid Partitions	Invalid Partitions	Valid Boundaries	Invalid Boundaries
Customer name	2 to 64 chars valid chars	< 2 chars > 64 chars invalid chars	2 chars 64 chars	1 chars 65 chars 0 chars



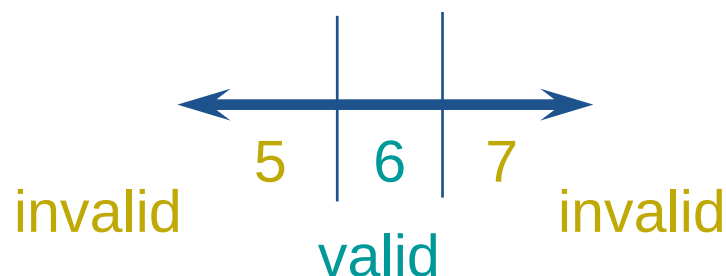
Account number

first character:

valid: non-zero

invalid: zero

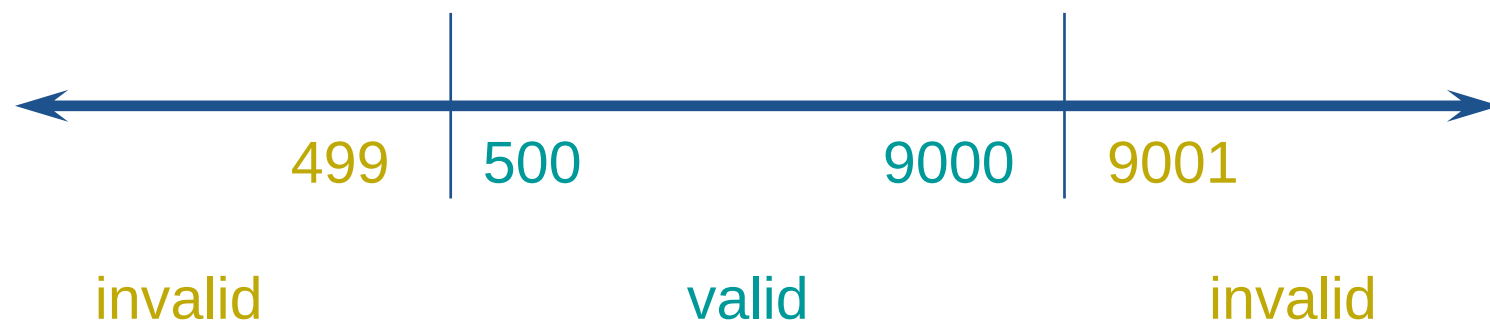
number of digits:



Conditions	Valid Partitions	Invalid Partitions	Valid Boundaries	Invalid Boundaries
Account number	6 digits 1 st non-zero	< 6 digits > 6 digits 1 st digit = 0 non-digit	100000 999999	5 digits 7 digits 0 digits



Loan amount



Conditions	Valid Partitions	Invalid Partitions	Valid Boundaries	Invalid Boundaries
Loan amount	500 - 9000	< 500 > 9000 0 non-numeric null	500 9000	499 9001

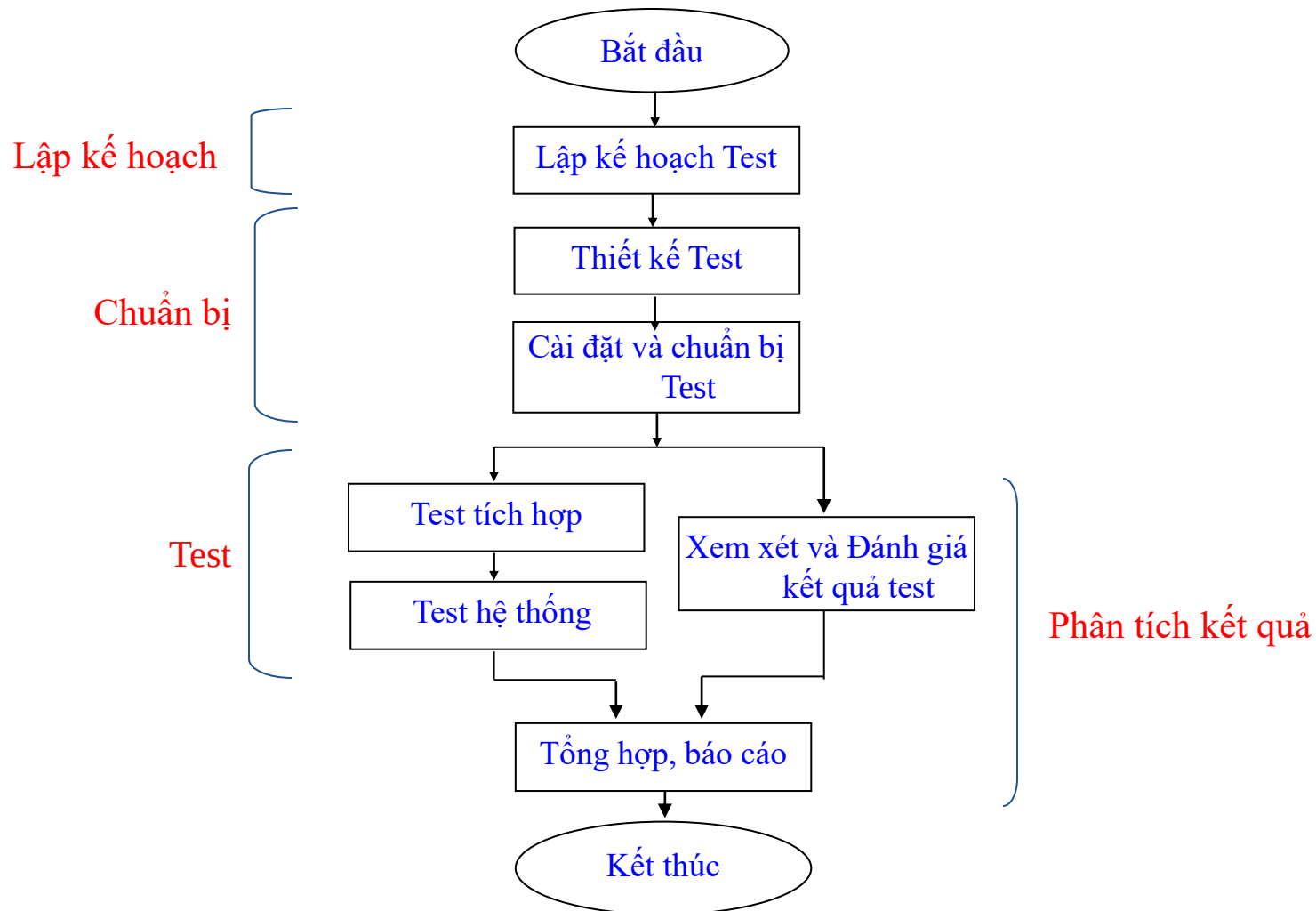


Condition template

Conditions	Valid Partitions	Tag	Invalid Partitions	Tag	Valid Boundaries	Tag	Invalid Boundaries	Tag
Customer name	2 - 64 chars valid chars	V1	< 2 chars > 64 chars invalid char	X1	2 chars 64 chars	B1	1 char 65 chars 0 chars	D1
		V2		X2		B2		D2
				X3				D3
Account number	6 digits 1 st non-zero	V3	< 6 digits > 6 digits 1 st digit = 0 non-digit	X4	100000 999999	B3	5 digits 7 digits 0 digits	D4
		V4		X5		B4		D5
				X6				D6
				X7				
Loan amount	500 - 9000	V5	< 500 >9000 0 non-integer null	X8	500 9000	B5	499 9001	D7
				X9		B6		D8
				X10				
				X11				
				X12				

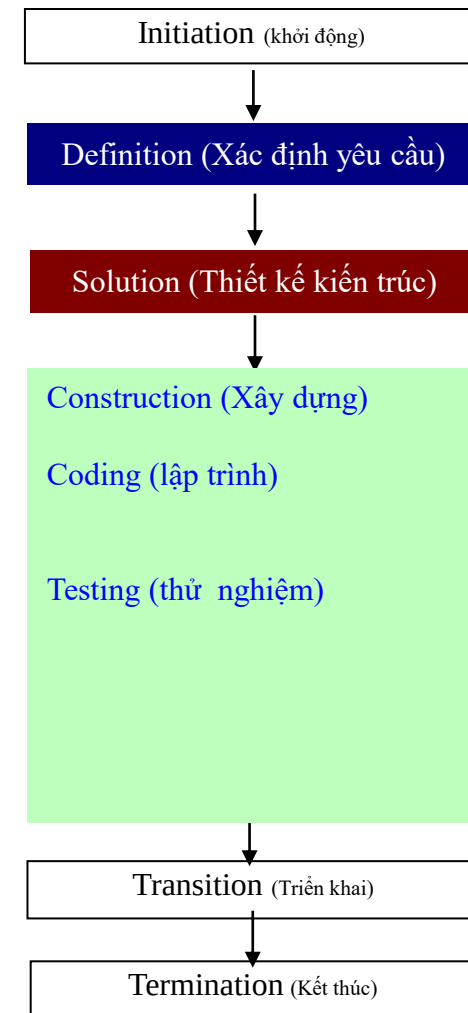
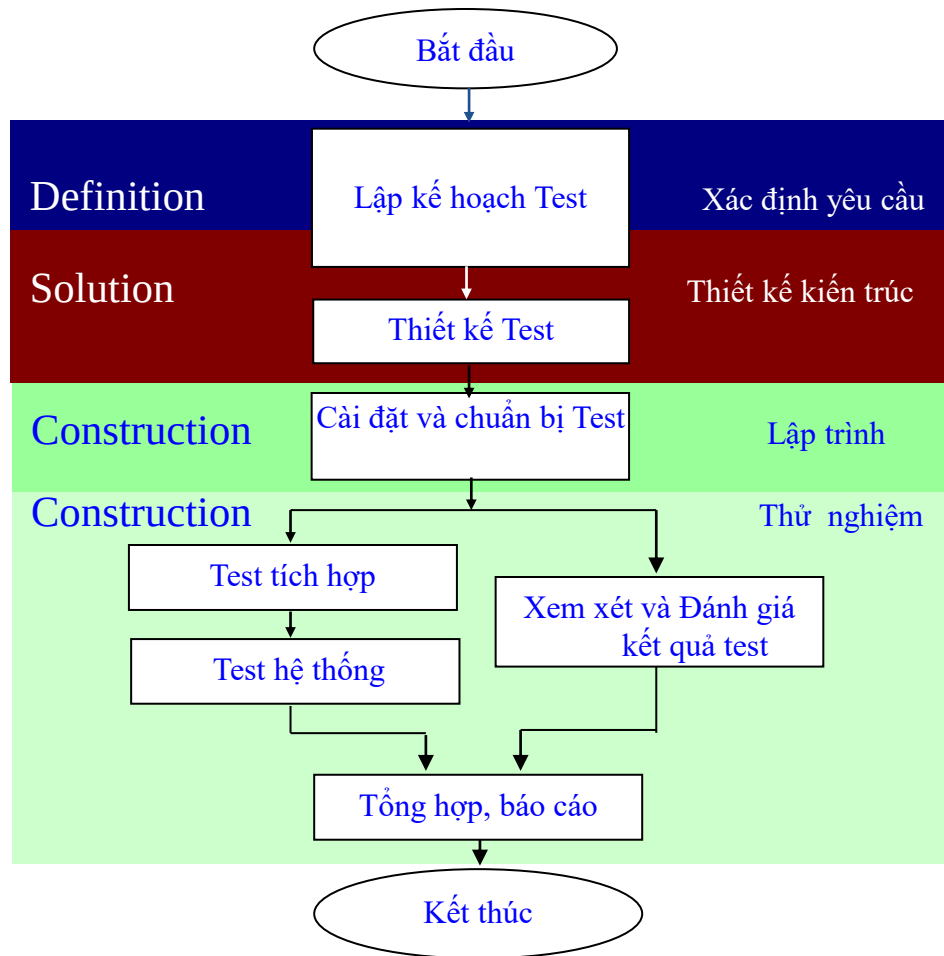


CÁC HOẠT ĐỘNG KIỂM THỬ



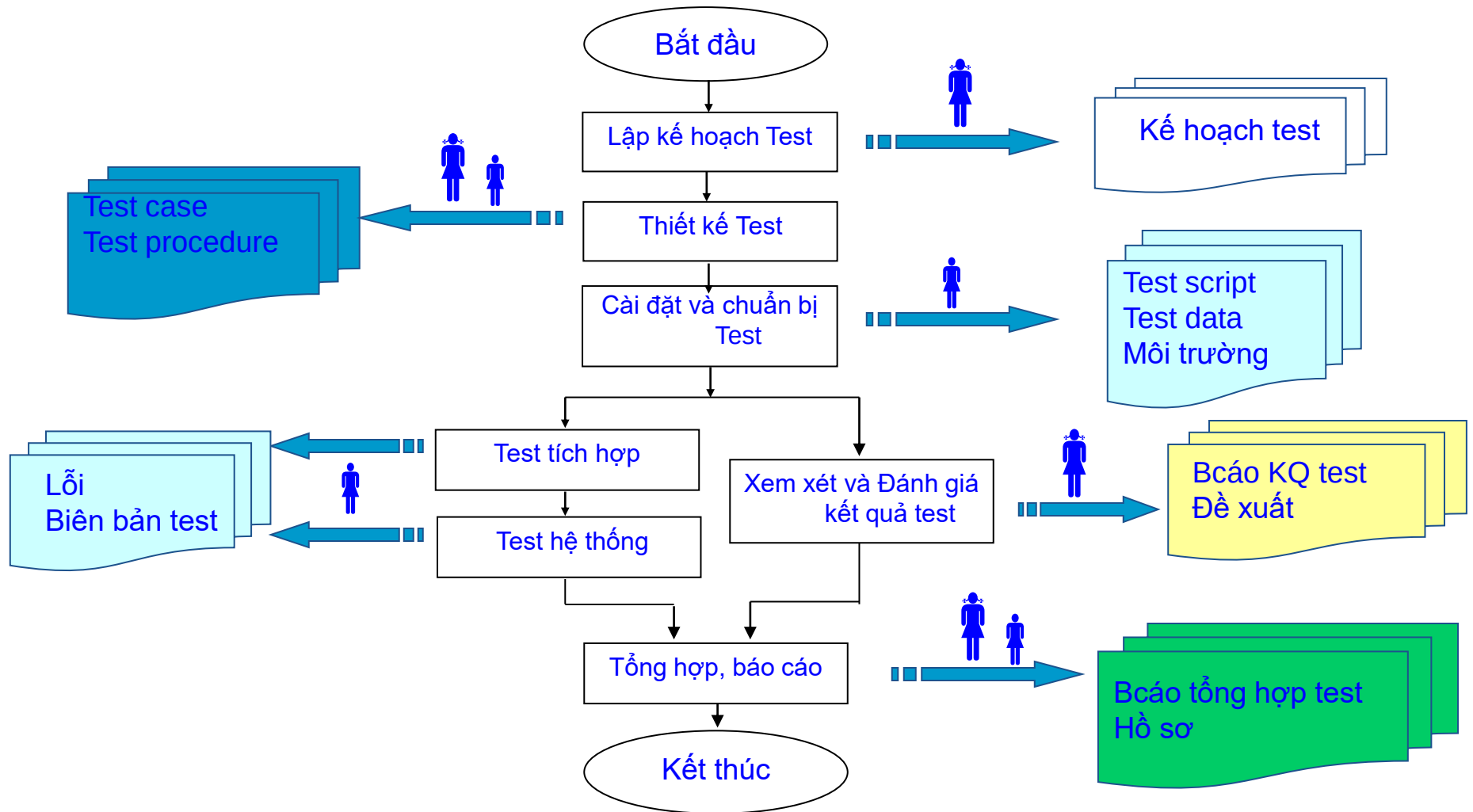


CÁC HOẠT ĐỘNG KIỂM THỬ





CÁC HOẠT ĐỘNG KIỂM THỬ





CÁC HOẠT ĐỘNG - Lập kế hoạch test

1. Xác định yêu cầu cho test

Các yêu cầu

- Dựa trên các sản phẩm phân tích
- Xác định: test cái gì?
- Xác định: giới hạn công việc, thời gian, nguồn lực

2. Đánh giá rủi ro và lập mức ưu tiên cho các yêu cầu

Các rủi ro

3. Xác lập chiến lược test

- Phương thức thực hiện
- Tiêu chuẩn hoàn thành việc test, đánh giá chất lượng sản phẩm
- Các yếu tố cần chú ý

TN Test

4. Xác định nguồn lực và môi trường

- Số người, kỹ năng
- Môi trường test: phần mềm, cứng...
- Công cụ test
- Dữ liệu test

5. Lập lịch trình test

TN Test

6. Tổng hợp thông tin, lập KH test

TN Test

7. Xem xét và thông qua KH test

- Xác định nguồn lực
- Lịch trình test
- Các mốc chính

QTDA



CÁC HOẠT ĐỘNG - thiết kế test

1. Lập danh sách các loại test, đảm bảo cho việc xác lập tính đúng đắn & thỏa mãn yêu cầu của sản phẩm

Danh

- Dựa trên các sản phẩm của:
 - Phân tích
 - bước lập KH test
- Xác định các test case
- Mô tả test case: vào, ra, điều kiện

2. Xây dựng tình huống test (test case)

Thiết
dụng

Tình huống kiểm thử

- Kiểm tra 1 chức năng của chương trình
- Các giá trị đầu vào $\xrightarrow{\text{Các điều kiện thực hiện}}$ Kết quả dự kiến

3. Xây dựng và tổ chức thủ tục test (test procedure)

Các t

CBT

4. Xem xét tình huống test và thủ tục test, đánh giá tỷ lệ yêu cầu của khách hàng (hoặc tình huống sử dụng) sẽ được test dựa trên thiết kế test đã lập

Đi

- Dựa vào: các test case
- Xác định các test procedure
- Xác lập mối quan hệ và thứ tự giữa
 - Các test procedure với nhau
 - Các test procedure - Test cases

5. Thông qua thiết kế Test

Thi

Thủ tục kiểm thử

- Các chỉ dẫn để thiết lập, thực hiện và đánh giá kết quả test
- Cho 1 hoặc nhiều tình huống test (test case)



Xây dựng tình huống test (test case)

- Một Test Case có thể coi là một tình huống kiểm tra, được thiết kế để kiểm tra một đối tượng có thỏa mãn yêu cầu đặt ra hay không. Một Test Case thường bao gồm 3 phần cơ bản:
 - Mô tả: đặc tả các điều kiện cần có để tiến hành kiểm tra.
 - Nhập: đặc tả đối tượng hay dữ liệu cần thiết, được sử dụng làm đầu vào để thực hiện việc kiểm tra.
 - Kết quả mong chờ: kết quả trả về từ đối tượng kiểm tra, chứng tỏ đối tượng đạt yêu cầu.



Design test cases

Test Case	Description	Expected Outcome		New Tags Covered
1	Name: John Smith Acc no: 123456 Loan: 2500 Term: 3 years	Term:	3 years	V1, V2, V3, V4, V5
		Repayment:	79.86	
		Interest rate:	10%	
		Total paid:	2874.96	
2	Name: AB Acc no: 100000 Loan: 500 Term: 1 year	Term:	1 year	B1, B3, B5,
		Repayment:	44.80	
		Interest rate:	7.5%	
		Total paid:	537.60	



Xây dựng tình huống test (test case)

- Xây dựng một test-case để kiểm thử form tạo mới thẻ độc giả

Form1

LAP THE DOC GIA MOI

HO TEN LOT TEN

SỐ NHÀ ĐƯỜNG QUẬN

NGÀY SINH ĐIỆN THOẠI

MDG NL MÃ ĐỘC GIẢ

NGÀY LẬP THE NGÀY HẾT HẠN



CÁC HOẠT ĐỘNG - Cài đặt và chuẩn bị test

Test script

- Các lệnh dùng để tự động hoá các thủ tục test
- Lập trình, sử dụng công cụ test tự động

Lập trình hoặc sinh tự động: = tool

- **Kiểm tra thử nghiệm**

1. Lập các test script để thực hiện các tình huống test/thủ tục test (nếu cần)

Test scripts

TN Test/
CBT

2. Chuẩn bị dữ liệu test

- **Tạo mới dữ liệu**
- **Tái sử dụng các dữ liệu cũ**

CBT

3. **Chuẩn bị môi trường**

CBT

4. Kiểm tra các công cụ test

Biên bản kiểm tra công cụ test

CBT

5. Xem xét môi trường, điều kiện và dữ liệu test

Môi trường và dữ liệu test
Kiểm tra

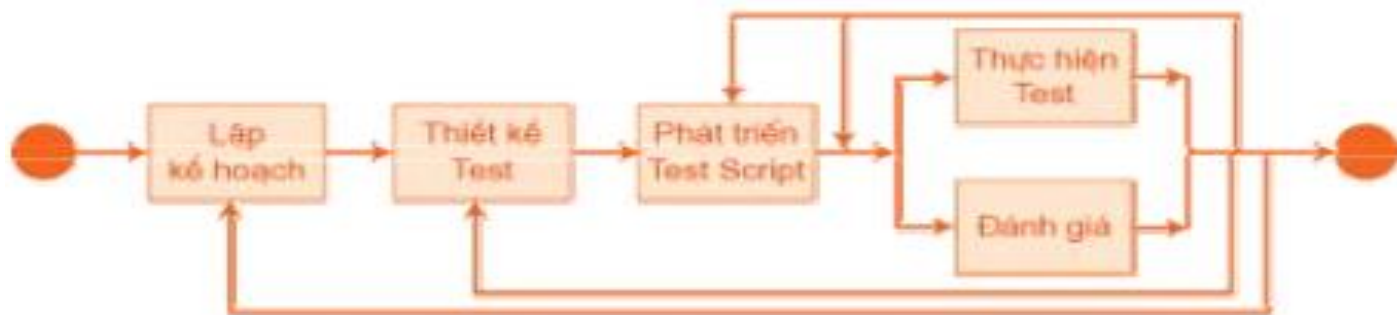
TN Test/
CBT

- **Phần mềm, phần cứng**
- **Các điều kiện khác**



Test Script

- Test Script là một nhóm mã lệnh dạng đặc tả kịch bản dùng để tự động hóa một trình tự kiểm tra, giúp cho việc kiểm tra nhanh hơn, hoặc cho những trường hợp mà kiểm tra bằng tay sẽ rất khó khăn hoặc không khả thi.
- Các Test Script có thể tạo thủ công hoặc tạo tự động dùng công cụ kiểm tra tự động.





CÁC HOẠT ĐỘNG - test tích hợp

		• Tài liệu • Gói phần mềm • ...
1. Nhận bàn giao với đội lập trình	Các sản phẩm cần nhận	
2. Cài đặt	Hệ thống test	• Dựa trên thiết kế test • Ghi nhận lỗi vào DMS
3. Thực hiện test và ghi nhận lỗi	Biên bản test	CBT
4. Xử lý lỗi	Danh sách lỗi phát hiện	TN Test, CBT
5. Xem xét các kết quả test và việc thực hiện khắc phục lỗi	Biên bản test	TN Test, QTDA, CBT, CBCL (nếu cần)
	• Phối hợp với đội lập trình • Test lại -> Ghi nhận lỗi mới	



CÁC HOẠT ĐỘNG - test hệ thống

1. Nhận bàn giao với đội lập trình	Các sản phẩm cần test được tiếp	CBT
2. Chỉnh sửa thiết kế test	• Kiểm tra và cập nhật: • Test case • Test procedure • Test script • Test data	TN Test
3. Cài đặt		CBT
4. Thực hiện test và ghi nhận lỗi		CBT
Danh sách lỗi phát hiện		
5. Xử lý lỗi	Biên bản test	TN Test, CBT
6. Xem xét các kết quả test thực hiện khắc phục lỗi	• Khi hệ thống thoả mãn các tiêu chuẩn đặt ra	TN Test,QTDA, CBT, CBCL (nếu cần)
7. Kết quả test được xem xét		Xác nhận sản phẩm đủ tiêu chuẩn phát hành



CÁC HOẠT ĐỘNG - xem xét và đánh giá kết quả test

1. Phân tích lỗi và đưa ra đề xuất	Báo cáo kết quả test Đề xuất	TN Test
2. Đánh giá tỷ lệ test, đánh giá mức độ đạt được các tiêu chí để hoàn thành test	Báo cáo kết quả test	TN Test
3. Xem xét báo cáo kết quả test	Báo cáo kết quả test được xem xét	QTDA, CBCL

- Dựa trên tỷ lệ YC được test/tổng số YC cần test

- Phân tích lỗi dựa:
 - mức độ lặp lại
 - độ nghiêm trọng
 - Thời gian sửa lỗi...



CÁC HOẠT ĐỘNG - tổng hợp, báo cáo

1. Tập hợp các dữ liệu, kết quả test	Dữ liệu, kết quả test	CBT
2. Lập báo cáo tổng hợp test	Báo cáo tổng hợp test	TN Test
3. Tổ chức lưu trữ tài liệu, hồ sơ	Hồ sơ, files	CBT



Công cụ tự động KT(Test tool)

- Hoạt động kiểm thử phần mềm (KTPM) đóng vai trò rất quan trọng, trong khi đó hoạt động này lại tiêu tốn và chiếm tỷ trọng khá lớn công sức và thời gian trong một dự án. Do vậy, nhu cầu tự động hoá qui trình KTPM cũng được đặt ra.
- Không phải mọi việc kiểm thử đều có thể tự động hóa. Việc dùng test tool thường được xem xét trong một số tình huống:
 - Không đủ tài nguyên: Khi số lượng tình huống kiểm thử (test case) quá nhiều mà các KTV không thể hoàn tất bằng tay trong thời gian cụ thể, hay trong kiểm thử hồi quy.
 - Kiểm tra khả năng vận hành phần mềm trong môi trường đặc biệt



Công cụ tự động KT(Test tool)

- KTTĐ có một số thuận lợi và khó khăn cơ bản khi áp dụng:
 - KTPM không cần can thiệp của KTV.
 - Giảm chi phí khi thực hiện kiểm tra số lượng lớn test case hoặc test case lặp lại nhiều lần.
 - Giả lập tình huống khó có thể thực hiện bằng tay.
 - Mất chi phí tạo các script để thực hiện KTTĐ.
 - Tốn chi phí dành cho bảo trì các script.
 - Đòi hỏi KTV phải có kỹ năng tạo script KTTĐ.
 - Không áp dụng được trong việc tìm lỗi mới của PM.



Công cụ tự động KT(Test tool)

- Để thành công trong KTTĐ chúng ta nên thực hiện các bước cơ bản sau:
 - Thu thập các đặc tả yêu cầu hoặc test case; lựa chọn những phần cần thực hiện KTTĐ.
 - Phân tích và thiết kế mô hình phát triển KTTĐ.
 - Phát triển lệnh đặc tả (script) cho KTTĐ.
 - Kiểm tra và theo dõi lỗi trong script của KTTĐ



Công cụ tự động KT(Test tool)

- Sử dụng công cụ kiểm thử tự động Selenium IDE để:
 - Demo kiểm thử màn hình đăng nhập API login của trang <https://tongkhogiasivn/> công ty cổ phần công nghệ RNET
 - Demo kiểm thử màn hình đăng nhập API login của Google



Các chuẩn kiểm thử phần mềm trong công nghiệp

- ISO/IEC/IEEE 29119 Là chuẩn được phát triển bởi ISO/IEC JTC1/SC7 Working Group 26(tương đương với chuẩn kiểm thử TCVN 12849:2020)
- Là tập hợp các tiêu chuẩn quốc tế về kiểm thử phần mềm. Có thể được sử dụng trong bất kỳ giai đoạn nào của vòng đời phát triển phần mềm hay bất kỳ tổ chức nào. Hiện có 6 tiêu chuẩn:
 - ISO/IEC/IEEE 29119-1: Mục đích của chuẩn này là để người dùng nắm được các hiểu biết và sử dụng được các chuẩn nằm trong tập chuẩn ISO/IEC/IEEE 29119. Nó giới thiệu tất cả các từ vựng và các ứng dụng của từng khái niệm đó trong thực tế. Đồng thời nó còn cung cấp cách ứng dụng các tiến trình, tài liệu, các kỹ thuật được định nghĩa trong bộ chuẩn này.



Các chuẩn kiểm thử phần mềm trong công nghiệp

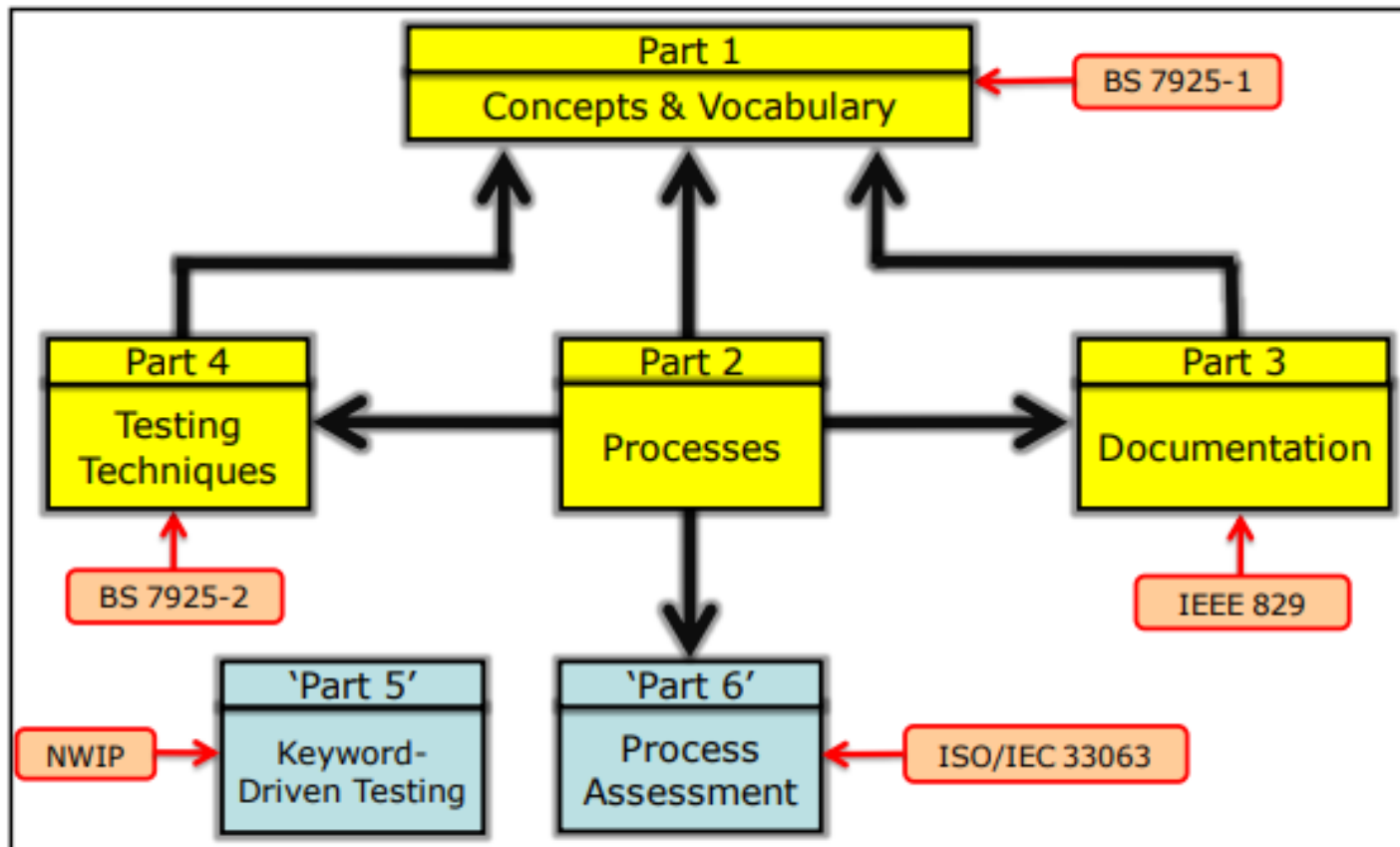
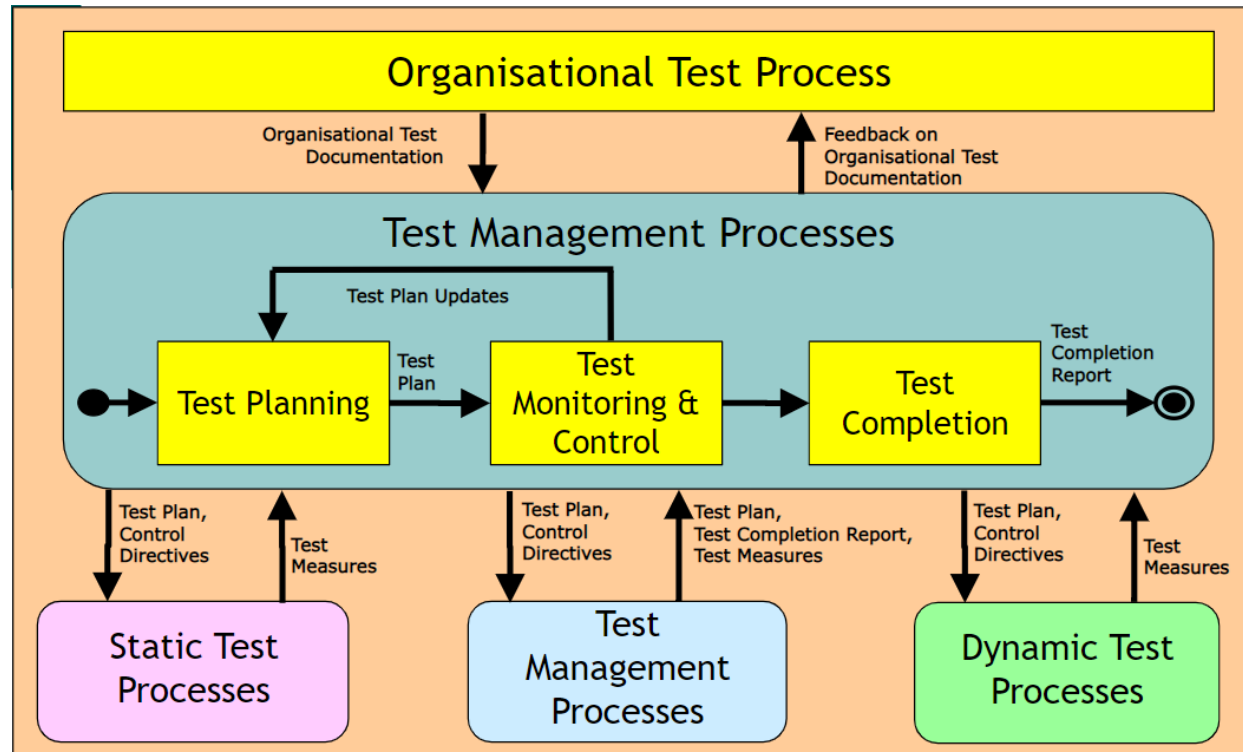


Figure 2: ISO/IEC 29119 Software and Systems Engineering Standards



Các chuẩn kiểm thử phần mềm trong công nghiệp

- ISO/IEC/IEEE 29119-2: Mục đích của chuẩn này là xác định một mô hình quy trình chung để kiểm thử phần mềm.





Các chuẩn kiểm thử phần mềm trong công nghiệp

Organisational Test Process

Test Management Processes

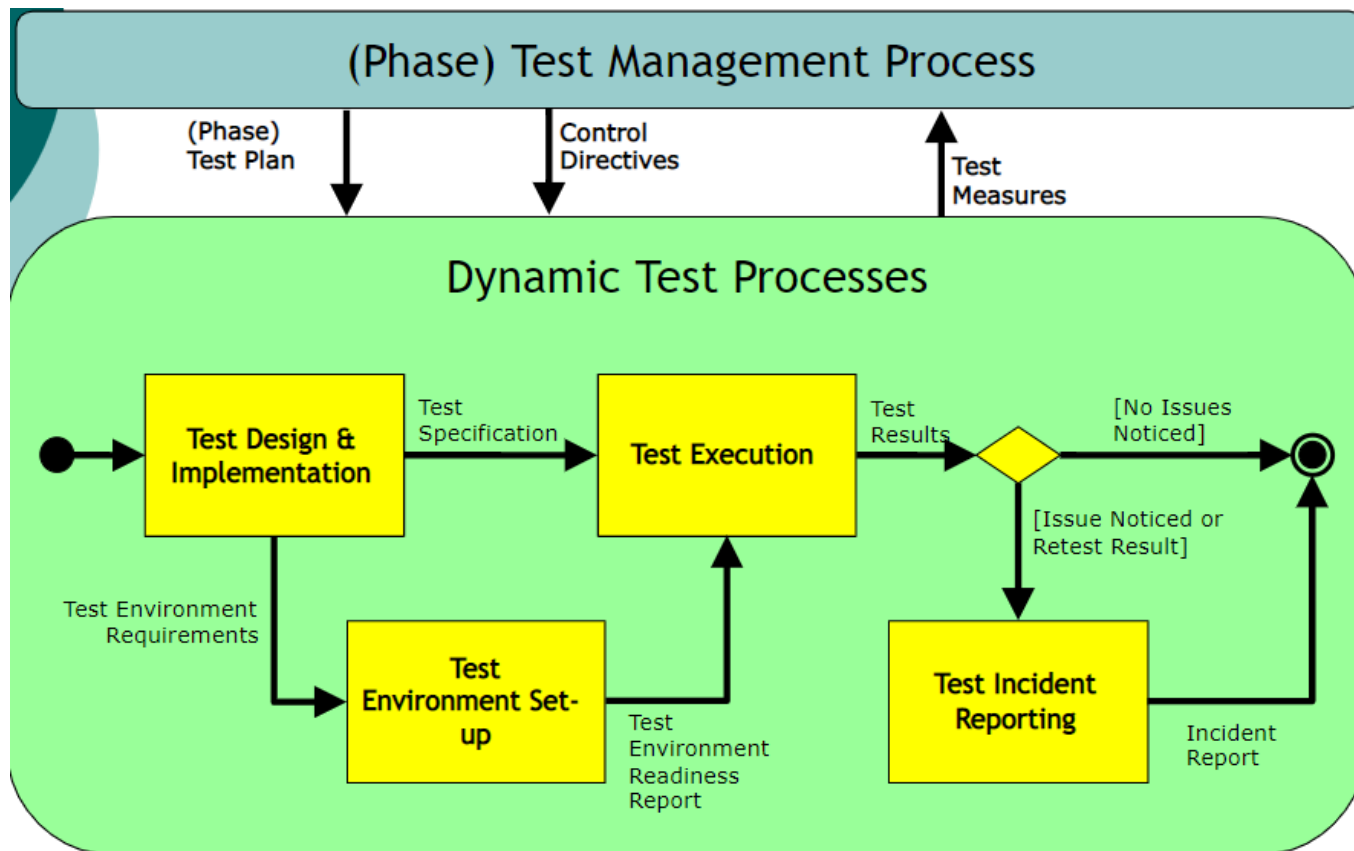
**Static Test
Processes**

**Dynamic Test
Processes**



Các chuẩn kiểm thử phần mềm trong công nghiệp

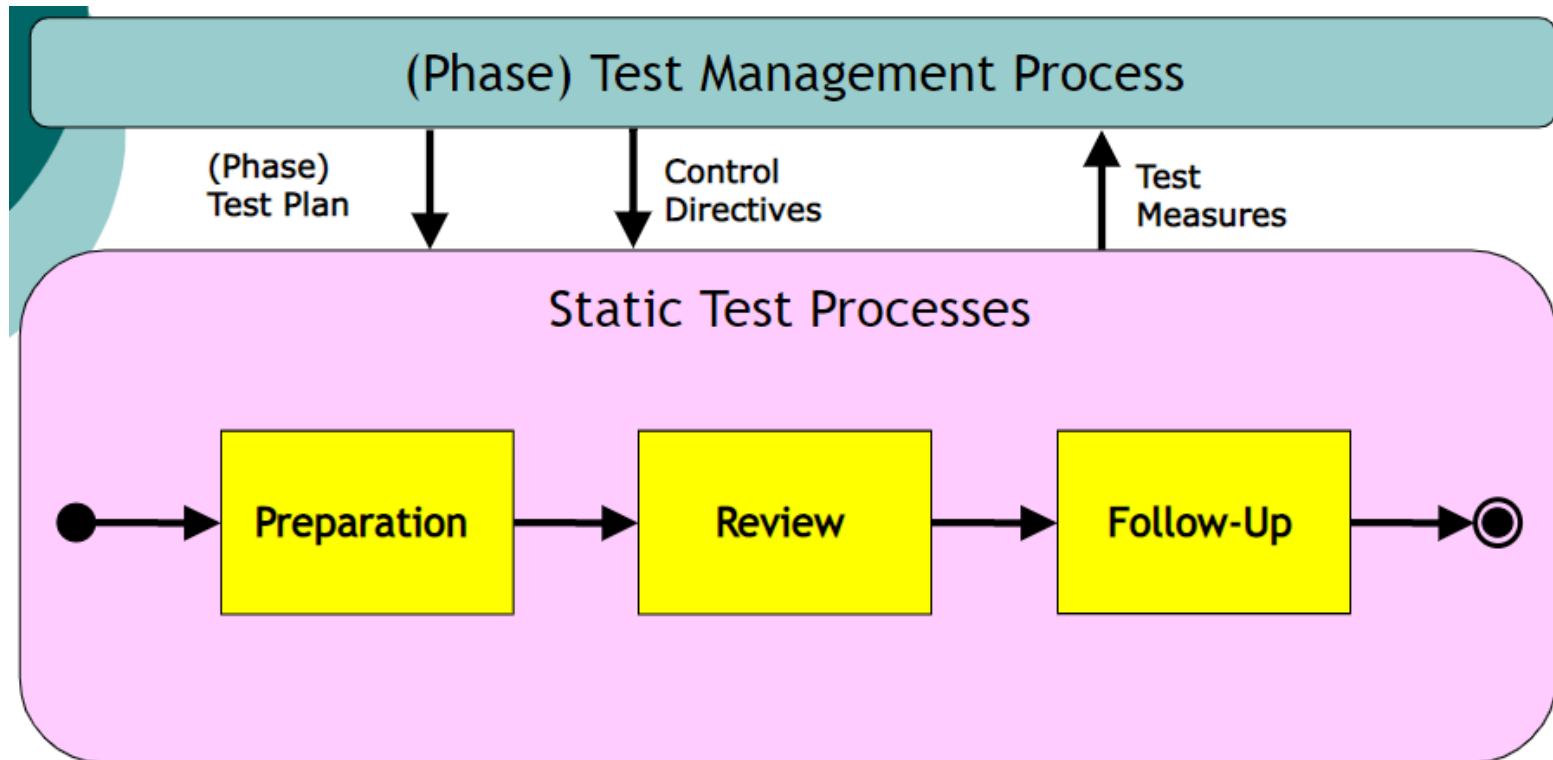
- Quy trình kiểm thử động





Các chuẩn kiểm thử phần mềm trong công nghiệp

- Quy trình kiểm thử tĩnh





Các chuẩn kiểm thử phần mềm trong công nghiệp

ISO/IEC/IEEE 29119-3:

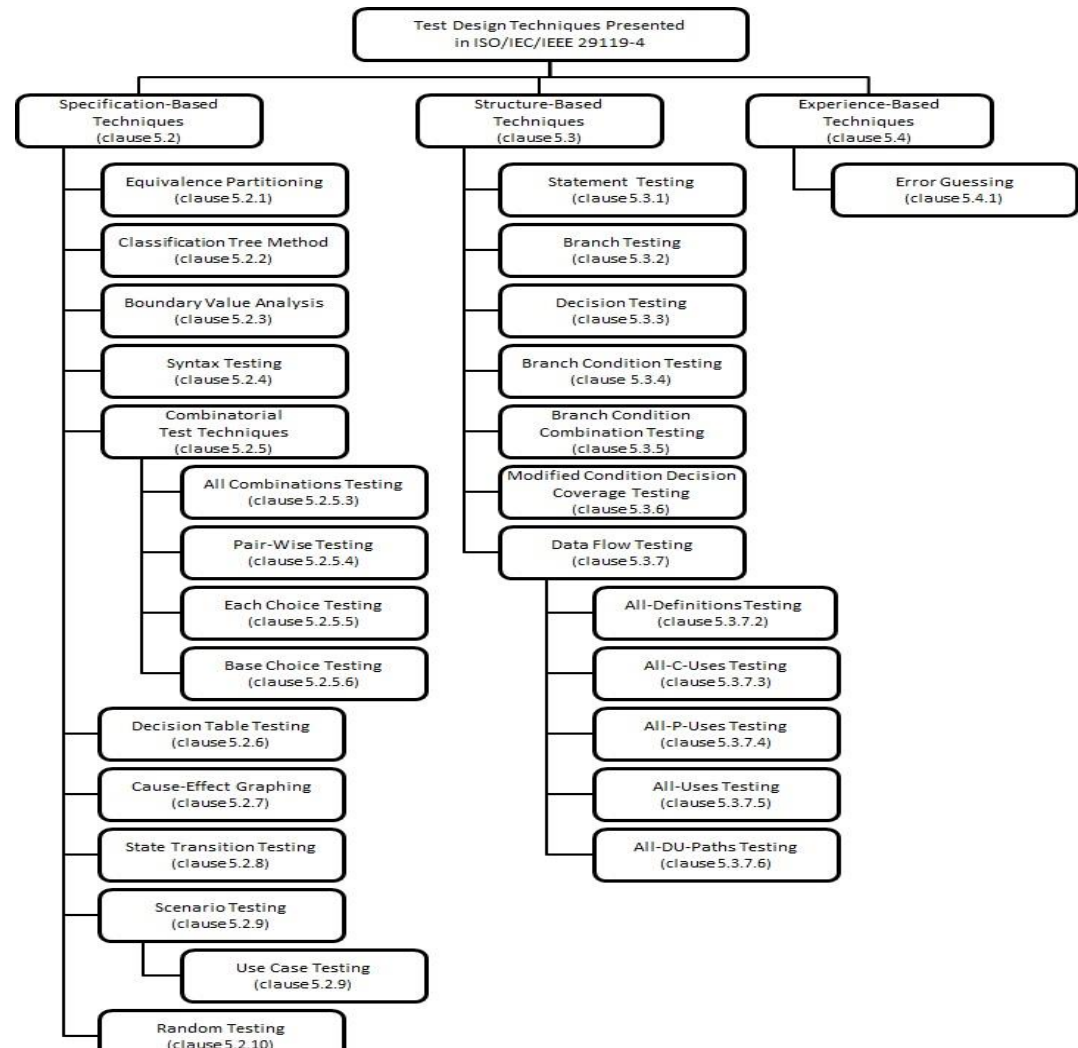
- Mục đích của chuẩn này là cung cấp các mẫu tài liệu kiểm thử. Mỗi mẫu tài liệu có thể được thay đổi nhằm phù hợp với nhu cầu riêng của từng tổ chức thực hiện triển khai các tiêu chuẩn.
- Chuẩn IEEE 829 đã khá nổi tiếng và được làm cơ sở cho tiêu chuẩn này. Có nghĩa là tiêu chuẩn này thay thế cho IEEE 829.
- Các tài liệu được định nghĩa bao gồm:
 - Tài liệu về quá trình tổ chức kiểm thử
 - Tài liệu về quá trình quản lý kiểm thử
 - Tài liệu về quá trình kiểm thử động, kiểm thử tĩnh



Các chuẩn kiểm thử phần mềm trong công nghiệp

ISO/IEC/IEEE 29119-4: Cung cấp chuẩn chung cho các kỹ thuật thiết kế kiểm thử

- Kỹ thuật kiểm thử dựa trên đặc tả.
- Kỹ thuật kiểm thử dựa trên cấu trúc.
- Kỹ thuật kiểm thử dựa trên kinh nghiệm.





Các chuẩn kiểm thử phần mềm trong công nghiệp

ISO/IEC/IEEE 29119-5:

- Mục đích nhằm xác định một tiêu chuẩn quốc tế hỗ trợ kiểm thử hướng Keyword (là một cách để mô tả testcase bằng cách sử dụng một bộ định sẵn các từ khóa-là các tên được liên kết với một tập hợp các hành động được yêu cầu để thực hiện một bước cụ thể trong testcase).
- Đây là cách sử dụng các từ khóa để mô tả thay thế cho các bước kiểm thử bằng ngôn ngữ tự nhiên. Do đó testcase trở nên dễ hiểu, dễ bảo trì và tự động hóa.



Các chuẩn kiểm thử phần mềm trong công nghiệp

ISO/IEC/IEEE 29119-6: Đánh giá quy trình kiểm thử

- Tiêu chuẩn này dựa trên tiêu chuẩn 2- quy trình kiểm thử và đang tiến triển tốt.
- Tiêu chuẩn này còn được gọi là ISO/IEC 33063

Một số chuẩn khác: (Từ khóa gợi ý) TMM, CMM/CMMi, ISO9000, ISO/IEC 9126, IEEE 829, IEEE 1061, IEEE 1050

...



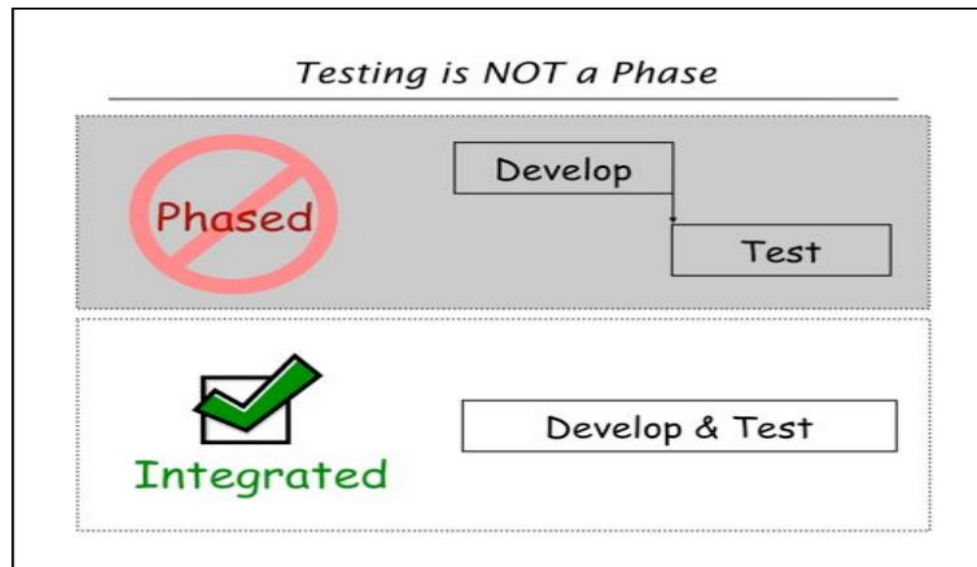
Kiểm thử phần mềm trong mô hình Agile

- Quan điểm kiểm thử của mô hình Agile:
 - Quan điểm kiểu truyền thống: Kiểm thử được xem là bước cuối kiểm tra chất lượng sản phẩm. Lỗi phát hiện sẽ làm chậm thời gian bàn giao sản phẩm.
 - Quan điểm dự án Agile: Xây dựng sản phẩm tốt ngay từ đầu. Sử dụng kiểm thử để phản hồi ngay khi phát triển (test driven).



Kiểm thử phần mềm trong mô hình Agile

- Mỗi liên hệ giữa tester và developer cần là sự cộng tác, tương hỗ lẫn nhau.
- Kiểm thử không còn là một giai đoạn của dự án. Nó cần được tham gia sâu vào quy trình phát triển từ sớm.





Kiểm thử phần mềm trong mô hình Agile

- Với dự án truyền thống, tester làm việc độc lập và chịu trách nhiệm với toàn bộ quá trình test.
- Với Agile, các hoạt động test được thực hiện với toàn bộ đội dự án. Để thực hiện được hết test thì cần thực hiện lặp lại qua các sprint.
- Rút ngắn vòng lặp phản hồi. Sử dụng nhiều test tự động. Do đó phát hiện vấn đề một cách nhanh chóng, giảm rủi ro, giảm việc phải làm lại.



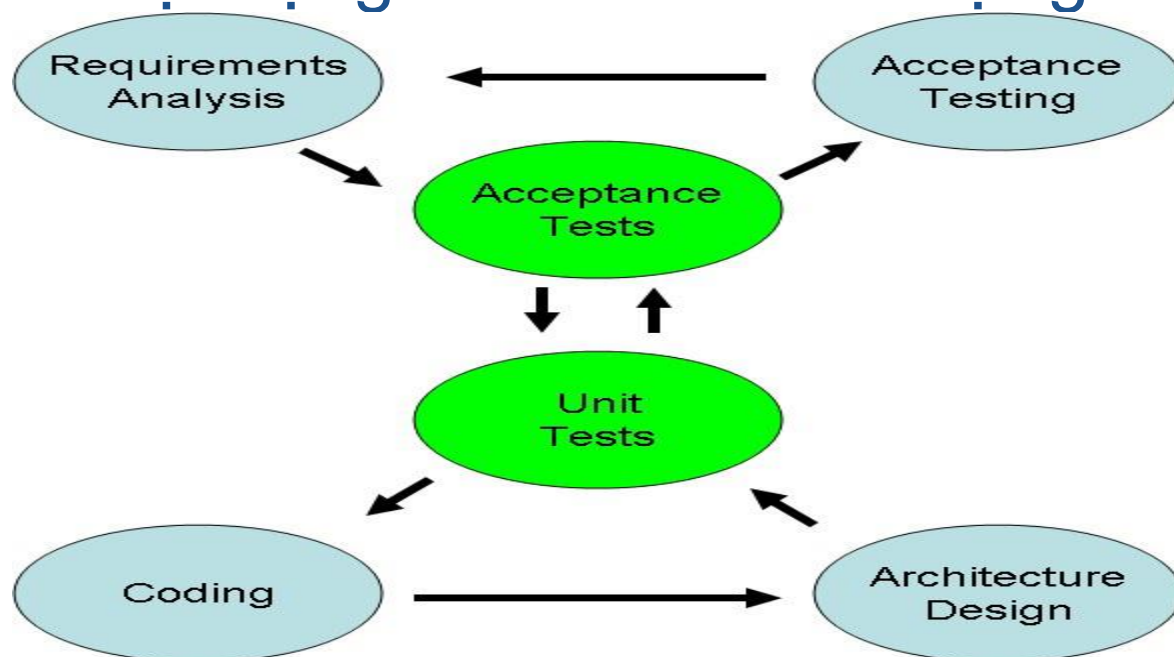
Kiểm thử phần mềm trong mô hình Agile

- Giảm lược tài liệu kiểm thử bằng cách:
 - Tái sử dụng checklist
 - Tập trung vào bản chất của các thử nghiệm chứ không phải các chi tiết ngẫu nhiên
 - Sử dụng các tài liệu hướng dẫn đơn giản
 - Nắm bắt những ý tưởng thử nghiệm trong điều lệ kiểm nghiệm thăm dò.



Kiểm thử phần mềm trong mô hình Agile

- Các giai đoạn kiểm thử tương ứng với các giai đoạn phát triển trong mô hình Agile:
 - Tiền phân đoạn
 - Xác minh yêu cầu
 - Các hoạt động đảm bảo chất lượng





Nội dung

1. Đảm bảo chất lượng phần mềm
2. Kiểm thử phần mềm
- 3. Bảo trì phần mềm



Bảo trì là gì?

- Định nghĩa: Bảo trì là công việc tu sửa, thay đổi phần mềm đã được phát triển (chương trình, dữ liệu, các loại tư liệu đặc tả, . . .) theo những lý do nào đó
- Các hình thái bảo trì: bảo trì để
 - Tu chỉnh
 - Thích hợp
 - Cải tiến
 - Phòng ngừa



Bảo trì để tu sửa

- Là bảo trì khắc phục những khiếm khuyết có trong phần mềm
- Một số nguyên nhân điển hình
 - Kỹ sư phần mềm và khách hiểu nhầm nhau
 - Lỗi tiềm ẩn của phần mềm do sơ ý của lập trình hoặc khi kiểm thử chưa bao quát hết
 - Vấn đề tính năng của phần mềm: không đáp ứng được yêu cầu về bộ nhớ, tệp .v..v. Thiết kế sai, biên tập sai . . .
 - Thiếu chuẩn hóa trong phát triển phần mềm (trước đó)



Bảo trì để tu sửa (tiếp)

- Kỹ nghệ ngược (Reverse Engineering):
dò lại thiết kế để tu sửa
- Những lưu ý
 - Mức trừu tượng
 - Tính đầy đủ
 - Tính tương tác
 - Tính định hướng



Bảo trì để thích hợp

- Là tu chỉnh phần mềm theo thay đổi của môi trường bên ngoài nhằm duy trì và quản lý phần mềm theo vòng đời của nó
- Thay đổi phần mềm thích nghi với môi trường: công nghệ phần cứng, môi trường phần mềm
- Những nguyên nhân chính:
 - Thay đổi về phần cứng (ngoại vi, máy chủ, . . .)
 - Thay đổi về phần mềm (môi trường): đổi OS
 - Thay đổi cấu trúc tệp hoặc mở rộng CSDL



Bảo trì để cải tiến

- Là việc tu chỉnh hệ phần mềm theo các yêu cầu ngày càng hoàn thiện hơn, đầy đủ hơn, hợp lý hơn
- Những nguyên nhân chính:
 - Do muốn nâng cao hiệu suất nên thường hay cải tiến phương thức truy cập tệp
 - Mở rộng thêm chức năng mới cho hệ thống
 - Cải tiến quản lý kéo theo cải tiến tư liệu vận hành và trình tự công việc
 - Thay đổi người dùng hoặc thay đổi thao tác



Bảo trì để phòng ngừa

- Là công việc tu chỉnh chương trình có tính đến tương lai của phần mềm đó sẽ mở rộng và thay đổi như thế nào
- Thực ra trong khi thiết kế phần mềm đã phải tính đến tính mở rộng của nó, nên thực tế ta ít gặp với phần mềm này.
- Có thể gặp bảo trì phòng ngừa trong tái cấu trúc thiết kế.



Bảo trì để phòng ngừa (tiếp)

- Mục đích: sửa đổi để thích hợp với yêu cầu thay đổi sẽ có của người dùng
- Thực hiện những thay đổi trên thiết kế không tường minh
- Hiểu hoạt động bên trong chương trình
- Thiết kế / lập trình lại
- Sử dụng công cụ CASE

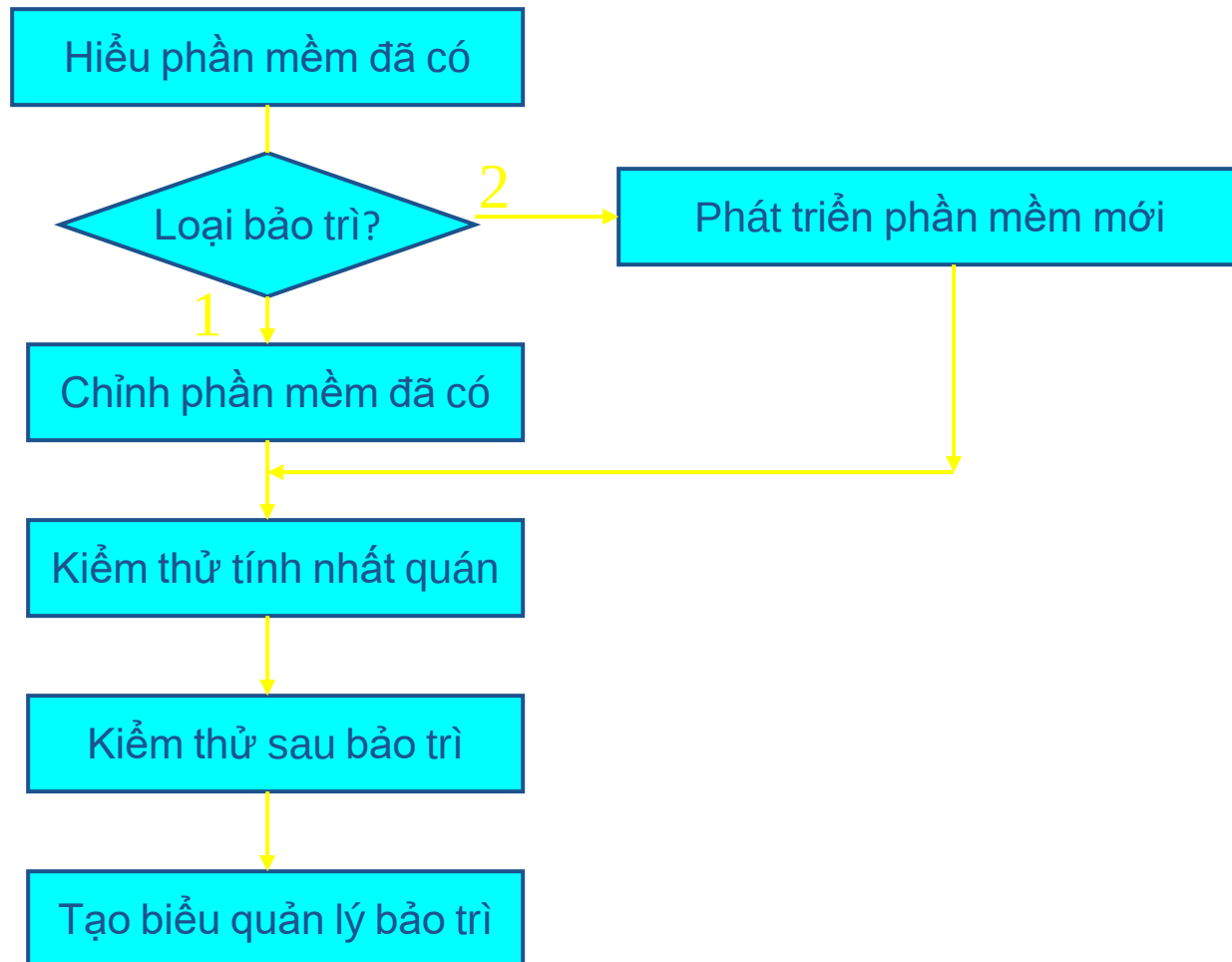


2. Trình tự nghiệp vụ bảo trì

- Quy trình bảo trì là gì ? Đó là quá trình trong vòng đời của phần mềm, cũng tuân theo các pha phân tích, thiết kế, phát triển và kiểm thử từ khi phát sinh vấn đề cho đến khi giải quyết xong
- Thao tác bảo trì: Gồm 2 loại
 - Tu chỉnh cái đã có (loại 1)
 - Thêm cái mới (loại 2)



Sơ đồ bảo trì





Hiểu phần mềm đã có

- Theo tài liệu nắm chắc các chức năng
- Theo tài liệu chi tiết hãy nắm vững đặc tả chi tiết, điều kiện kiểm thử, . . .
- Dò đọc chương trình nguồn, hiểu trình tự xử lý chi tiết của hệ thống



Tu sửa phần mềm đã có

- Bảo trì chương trình nguồn, tạo các môđun mới và dịch lại
- Thực hiện kiểm thử unit và tu chỉnh những mục liên quan có trong tư liệu đặc tả
- Chú ý theo sát tác động của môđun được sửa đến các thành phần khác trong hệ thống



Phát triển phần mềm mới

- Khi thêm chức năng mới phải phát triển chương trình cho phù hợp với yêu cầu
- Cần tiến hành từ thiết kế, lập trình, gỡ lỗi và kiểm thử unit
- Phản ánh vào giao diện của phần mềm (thông báo, phiên bản, . . .)



Kiểm chứng tính nhất quán bằng kiểm thử kết hợp

- Đưa đơn vị (unit) đã được kiểm thử vào hoạt động trong hệ thống
- Điều chỉnh sự tương thích giữa các môđun
- Dùng các dữ liệu trước đây khi kiểm thử để kiểm thử lại tính nhất quán
- Chú ý hiệu ứng làn sóng trong chỉnh sửa



Kiểm tra khi hoàn thành bảo trì

- Kiểm tra nội dung mô tả có trong tư liệu đặc tả
- Cách ghi tư liệu có phù hợp với mô tả môi trường phần mềm mới hay không ?



Lập biểu quản lý bảo trì

- Cần quản lý tình trạng bảo trì
- Lập biểu quản lý tình trạng bảo trì
 - Ngày tháng, giờ
 - Nguyên nhân
 - Tóm tắt cách khắc phục
 - Chi tiết khắc phục, hiệu ứng làn sóng
 - Người làm bảo trì
 - Số công



3. Những vấn đề lưu ý để bảo trì

Phương pháp cải tiến thao tác bảo trì:

- Sáng kiến trong quy trình phát triển phần mềm
- Sáng kiến trong quy trình bảo trì phần mềm
- Phát triển những kỹ thuật mới cho bảo trì



Sáng kiến trong quy trình phát triển phần mềm

- (1) Chuẩn hóa mọi khâu trong phát triển phần mềm
- (2) Người bảo trì chủ chốt tham gia vào giai đoạn phân tích và thiết kế
- (3) Thiết kế dễ để bảo trì



Sáng kiến trong quy trình bảo trì phần mềm

- (1) Sử dụng các công cụ hỗ trợ phát triển phần mềm
- (2) Chuẩn hóa thao tác bảo trì và môi trường bảo trì
- (3) Lưu lại những thông tin lịch sử bảo trì
- (4) Dự án nên cử một người chủ chốt của mình làm công việc bảo trì sau khi dự án kết thúc giai đoạn phát triển



Phát triển những kỹ thuật mới cho bảo trì

- Công cụ phần mềm hỗ trợ bảo trì
- Cơ sở dữ liệu cho bảo trì
- Quản lý tài liệu, quản lý dữ liệu, quản lý chương trình nguồn, quản lý dữ liệu thử, quản lý lịch sử bảo trì
- Trạm bảo trì tính năng cao trong hệ thống mạng lưới bảo trì với máy chủ thông minh



Bảo trì chương trình xa lạ

- Chương trình xa lạ?
- Phải làm gì với các chương trình xa lạ?
 1. Nghiên cứu, tìm hiểu trước khi bảo trì. Cố gắng có được nhiều thông tin nền tảng nhất có thể.
 2. Cố gắng quen thuộc với luồng điều khiển của toàn bộ chương trình, bỏ qua chi tiết mã hóa ban đầu. Sẽ rất hữu ích nếu bạn tự vẽ ra được sơ đồ cấu trúc và sơ đồ khối mức cao nếu chưa có.



Bảo trì chương trình xa lạ (tiếp)

3. Ước lượng tính hợp lý của tài liệu hiện có, đưa thêm vào lời giải thích của bạn vào bản in chương trình gốc nếu bạn nghĩ chúng có ích.
4. Dùng các bản in tham khảo chéo tốt, các bảng ký hiệu và các trợ giúp khác mà nói chung do trình biên dịch hay hợp dịch cung cấp.
5. Thay đổi chương trình với sự thận trọng lớn nhất. Hãy tôn trọng các định dạng chương trình nếu có thể được. Hãy chỉ ra trên bản in những lệnh nào mà bạn đã thay đổi



Bảo trì chương trình xa lạ (tiếp)

6. Đừng hủy bỏ mã lệnh trừ khi bạn chắc chắn nó không được dung tới.
7. Đừng cố dung chung biến tạm thời và bộ nhớ làm việc đã có trong chương trình, hãy thêm vào các biến riêng của mình để tránh rắc rối.
8. Hãy giữ các bản ghi chi tiết về hoạt động và kết quả bảo trì.
9. Tránh sự thôi thúc mạnh mẽ “đập đi xây lại”
10. Đừng xen lẫn việc kiểm tra lỗi.



Thảo luận

